

Scalable Distributed Key Generation for Blockchains

Aniket Kate
Supra Research/Purdue University
aniket@purdue.edu

Hamza Saleem
Supra Research
hamzasaleemzpr@gmail.com

Pratyay Mukherjee
Hashgraph
pratyay85@gmail.com

Nibesh Shrestha
Supra Research
nibeshrestha2@gmail.com

Pratik Sarkar
Supra Research
iampratiksarkar@gmail.com

David Yang
Supra Research
y.yang@supra.com

ABSTRACT

Distributed key generation (DKG) is a foundational building block for designing efficient threshold cryptosystems, which are crucial components of blockchain ecosystems. Existing DKG protocols address the problem in a standalone setting, focusing on establishing the final DKG public key and individual secret keys among the participating parties. This work focuses on DKG primitives for use over blockchain, where the final DKG public key must be available on-chain, enabling on-chain smart contracts to seamlessly execute threshold cryptographic verifications. We observe that existing standalone DKG designs do *not* sufficiently exploit the presence of blockchain, leaving substantial scope for improvement in performance.

In this work, we design the first discrete-log-based DKG protocol tailored for use over blockchain, leveraging the blockchain’s built-in consensus mechanism to realize DKG efficiently. Interestingly, the use of blockchains enables us to solve DKG while tolerating up to one-half Byzantine faults even in non-synchronous settings. Our protocol is asynchronous, allowing it to operate independently of the network’s timing assumptions, with the exact network model depending on the destination blockchain.

Our solution further utilizes an associated random beacon to select smaller committees and achieves a DKG protocol with sub-cubic communication complexity, sub-quadratic computation complexity, and minimal on-chain storage. Notably, our protocol employs a single invocation of consensus and can terminate in just eleven communication rounds in the good case when deployed on an optimal latency partially synchronous blockchain. Our experiments show that our protocol terminates faster than state-of-the-art standalone protocols, with similar bandwidth overhead for committee members and significantly reduced bandwidth for other parties. Additionally, our protocol benefits from higher CPU resources—when deployed on machines with 32 vCPUs, it completes in approximately 6.5 seconds in the optimistic case, even for larger systems with 256 nodes.

1 INTRODUCTION

Distributed key generation (DKG) [48] aims to establish a public key and its corresponding secret keys among a group of n parties in a secret-sharing fashion such that no single party knows the shared secret key. DKG protocols reduce trust assumptions in cryptographic systems such as threshold signatures [15] and threshold encryption schemes [34]. These threshold cryptosystems have diverse applications, including implementing random beacons [19, 37], simplifying consensus protocols [76, 81], enabling

secure multiparty computation [55, 56], and delegating secret management to multiple semi-trusted authorities [39, 62].

As threshold cryptography gains traction in blockchains and distributed systems, DKG has emerged as a topic of significant interest. A substantial body of research has focused on addressing the DKG problem efficiently, with efforts dedicated to reducing communication and round complexity while ensuring optimal resilience across various network settings [1, 3, 11, 29, 32, 42, 48, 57, 77]. However, to the best of our knowledge, these protocols focus on solving the DKG problem in a *standalone* manner, where the primary goal is to agree on the final DKG public key and corresponding individual secret keys among n parties.

Distributed key generation for use over a blockchain. With the growing adoption of blockchains, several on-(block)chain applications (e.g., generating or/and verifying randomness [37, 50, 54] and state commitments) demand the ability to validate threshold signatures directly on-chain. In particular, on-chain smart contracts often require access to the final DKG public key to facilitate the verification of threshold signatures within the blockchain. To support these applications, it is essential to extend the DKG primitive by defining *distributed key generation for use over blockchains*, which mandates that the final DKG public key generated through the DKG process be recorded on-chain—this is necessary for on-chain verification of threshold signatures, beacons, etc. Moreover, the DKG protocol should not impose any additional network assumptions on the (destination) blockchain that uses it. In particular, the design must be asynchronous to ensure that the network assumptions of the employed blockchain system remain unchanged.

A straightforward approach to solve this problem is to first execute an existing standalone asynchronous DKG (ADKG) protocol and subsequently publish the signed DKG public key on-chain. The blockchain then performs a total ordering of the posted signed DKG public keys, allowing a common final DKG public key, signed by at least $f + 1$ parties, to be used on-chain, where f is the maximum number of Byzantine parties.

While being conceptually simple, this approach incurs significant overhead in terms of communication, computation, and round complexity, and it also restricts the protocol’s fault tolerance to $f < n/3$. Specifically, the participating parties must first execute a standalone ADKG protocol, which requires a separate (asynchronous) consensus instance with $n > 3f$ to agree on a common set of parties that have correctly shared their secrets. Following this, when the signed DKG public key is posted on-chain, the process involves yet another consensus instance, as the blockchain nodes order the blocks. Overall, this straightforward approach inherently involves two separate consensus instances, which, as we discover, is

Table 1: Comparison of High-Threshold DKGs for use over Blockchains

	Res.	Off-chain Communication	On-chain Storage	Off-chain Computation	Round	Setup
Kokoris et al. [61]	1/3	$O(\kappa n^4)$	$O(\kappa n)$	$O(n^3)$	$E(O(n))$	PKI
Das et al. [32]	1/3	$O(\kappa n^3)$	$O(\kappa n)$	$O(n^2)$	$E(O(\log n))$	PKI
Das et al. [29]	1/3	$O(\kappa n^3)$	$O(\kappa n)$	$O(n^2)$	$E(O(\log n))$	PKI
Abraham et al. [3]	1/3	$O(\kappa n^3)$	$O(\kappa n)$	$O(n^2)$	$E(O(1))$	PKI
Zhang et al. [83]	1/3	$O(\kappa n^4)$	$O(\kappa n)$	$O(n^3)$	$E(O(2^n))$	-
Kate et al. [59]	1/2	-	$O(\kappa n^2)$	$O(n^2)$	5	PKI
Cascudo et al. [22]	1/3	$O(\kappa n^2)$	$O(\kappa n)$	$O(n^2)$	$E(O(1))$	PKI+SS
Feng et al. [44]	1/3	$O(\kappa n^2)$	$O(\kappa n)$	$O(n^2)$	$E(O(1))$	PKI+SS
This work	1/2	$O(\kappa n^3 + n_a n_c^3)$	$O(\kappa n_a + n_a n_c)$	$O(n^2)$	10	PKI+RB
This work	1/3	$O(\kappa n_c^2 n + n_a n_c^3 + \kappa n^2)$	$O(\kappa n_a + n_a n_c)$	$O(n_c)/O(n_c n)^*$	11	PKI+RB

In this table, we only compare DKG protocols that generate scalar secret keys. Our protocol leverages random beacons to select smaller committees and improve efficiency. We argue that, even with access to random beacons and blockchain, existing DKG protocols cannot achieve the same level of efficiency as our protocol, as detailed in Section 2. **Res.** implies resilience. **Comm.** implies communication. **E(.)** implies “in expectation”. **SS** denotes a silent setup phase, which incurs $O(\kappa n^3)$ communication overhead. **RB** implies random beacon. * $O(n_c)/O(n_c n)$ implies that parties outside the clan perform $O(n_c)$ computations, while parties within the clan perform $O(n_c n)$ computations.

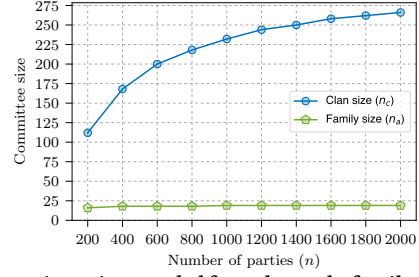
unnecessary. This redundancy not only adds substantial communication overhead and increases round complexity but also limits the resilience of the DKG process to tolerate at most $f < n/3$ Byzantine faults, as opposed to up to the best possible $f < n/2$ Byzantine faults (as we demonstrate) for DKG/threshold cryptography.

One may also think about using the recent non-interactive publicly verifiable secret sharing (PVSS) schemes [22, 47, 51, 59] to realize DKG for use over a blockchain. In this method, every party posts its PVSS vector onto the blockchain, and the first set of $f + 1$ valid PVSS vectors can be used to establish the DKG¹. This method is efficient in terms of off-chain communication and can tolerate up to $n/2$ Byzantine parties within the DKG committee. However, each PVSS vector has a size of $\Omega(\kappa n)$, which leads to the blockchain needing to store at least $O(\kappa n^2)$ information, plus also incurs $O(n^2)$ computation, where κ is the security parameter and n is the number of parties in the DKG committee. Overall, this method does not scale due to the high gas fees associated with storing and verifying $O(n^2)$ data on the blockchain.

Finally, publicly verifiable and unpredictable random beacons [37, 54, 64, 71] are readily available as a service and can be used to construct efficient DKG protocols (as in prior work [43]). For example, these random beacons can be utilized to select smaller committees of dealers who perform the DKG, enhancing the efficiency significantly. However, we observe that even with access to random beacons and blockchain, existing standalone DKG protocols remain inefficient (as explained later in Section 2 when we provide more context). This naturally begets the following important question:

Leveraging access to blockchain and random beacons can we design a significantly more efficient discrete-log-based DKG protocol for blockchains?

¹We note that non-interactive PVSS approaches [22, 47, 51, 59] have been utilized to realize DKG assuming broadcast channels. We observe that, due to the non-interactive nature of these schemes, they can also be employed to realize DKG in a non-synchronous blockchain setting, despite the blockchain offering weaker atomic broadcast [24] guarantees compared to broadcast channels: a broadcast channel should ensure that every honest input appears on the channel in within a know time bound, which we cannot ensure on any popular blockchain without setting time-bound to be extremely large. For the lightning network over Bitcoin, they set this time-bound to be in days.

**Figure 1: Committee sizes needed for a clan and a family as a function of n ensuring a failure probability $< 10^{-9}$.**

We affirmatively address this question by introducing the first discrete-log based DKG protocol tailored for blockchain equipped with a randomness beacon. By using the beacon, we randomly sample sub-committees and design efficient DKG protocols under two fault tolerance settings: super-honest majority ($f < n/3$) and honest majority ($f < n/2$). In the super-honest majority setting, our protocol achieves $O(\kappa n_c^2 n + n_a n_c^3 + \kappa n^2)$ communication and $O(\kappa n_a + n_a n_c)$ on-chain storage. Here n_c denotes the size of an honest-majority sub-committee (termed *clan*) sampled from n parties, while n_a refers to a smaller sub-committee (termed *family*) with at least one honest party, also sampled from n parties. Comparative committee sizes are shown in Figure 1. In this setting, the computational load is primarily handled by the clan, performing $O(n_c n)$ computation, while the remaining parties carry out only $O(n_c)$ computations each. Importantly, our protocol requires only a single invocation of the consensus mechanism—realized by blockchain and completes in 11 rounds in the good-case² when using an optimal latency partially synchronous blockchain [35, 75]. This setting is tailored for applications like consensus and on-chain randomness, where a super-honest majority assumption is necessary. In the honest majority setting, our protocol achieves $O(\kappa n^3 + n_a n^3)$ communication, $O(\kappa n_a + n_a n)$ on-chain storage, $O(n^2)$ off-chain computation and terminate in 10 rounds in the good-case when using an optimal-latency partially synchronous blockchain protocol [35, 75].

Finally, our protocol generates scalar secret keys and is compatible with off-the-shelf threshold cryptosystems [16, 34]. We present a comparison of our protocol with existing approaches for DKG over blockchain in Table 1.

Implementation and evaluation. We implemented and evaluated our protocol, comparing its performance to the state-of-the-art standalone ADKG by Das et al. [29] when deployed over a blockchain. At $n = 256$, our protocol terminates approximately 3.3× faster in optimistic scenarios, where all parties respond promptly, and about 17% faster during pessimistic executions with up to f Byzantine failures when deployed on machines with 2 vCPUs. With higher computational resources, the efficiency improves significantly—on machines with 32 vCPUs, our protocol completes in approximately 6.5 seconds in optimistic conditions, demonstrating its scalability. Additionally, the bandwidth consumption for clan parties is approximately 3.8× lower in optimistic cases and comparable in pessimistic cases. We also computed the Ethereum gas cost for posting data

²Good-case implies the leader of the underlying consensus protocol is honest, and the network is synchronous. [6]

on-chain when using Ethereum as a blockchain and found that our protocol incurs significantly lower gas costs compared to those of Kate et al.[59], and Das et al.[29] due to minimal on-chain storage required by our protocol.

2 OVERVIEW

Standalone DKG and strawman alternatives. Existing standalone ADKG protocols [3, 29, 32, 83] commonly employ n parallel asynchronous verifiable/complete secret sharings (AVSS/ACSS) [40, 69, 82]. An ACSS scheme allows a dealer to distribute secret shares to parties such that (i) each party can verify the consistency of their received share with the shared secret and (ii) a subset of parties can reconstruct the secret by combining their shares. These protocols typically utilize weaker consensus primitives, such as reliable broadcast [18] (RBC), to ensure that the dealer distributes a consistent secret to all honest parties and that all honest parties receive valid secret shares.

After the sharing phase, parties must agree on a common set of qualified parties Q (which includes at least one honest party) that have correctly shared their secrets. This problem is referred to as the agreement on a core set (ACS) problem [1] and requires $n > 3f$. Once the ACS phase is completed, it ensures that all honest parties have obtained valid secret shares corresponding to the secrets shared by the parties in Q . Subsequently, the final public key and secret keys are derived from the aggregated secret shares of the parties in Q .

This standard approach incurs high overhead: even with state-of-the-art ACSS [7, 8], n parallel instances cost $O(\kappa n^3)$ in communication, and the ACS phase—requiring n parallel Byzantine Agreement (BAs)—also incurs at least $\Omega(\kappa n^3)$ communication. We observe that even with blockchain and random beacons, standalone DKG protocols do not match our efficiency. For instance, consider a strawman solution where we sample a clan of dealers via a random beacon and use state-of-the-art ACSS during the sharing phase. This approach still incurs at least $O(\kappa n_c n^2)$, excluding ACS overhead.

We further observe that non-interactive PVSS-based solutions (see Table 1) are also unable to match the efficiency of our protocol, even with access to random beacons and blockchain. While these protocols can tolerate up to $f < n/2$ Byzantine faults, electing a clan is not feasible in these protocols, resulting in no improvement in on-chain storage.

2.1 Our Approach

Efficient DKG via random sub-sampling. In most standalone DKG protocols, every party in the system acts as a dealer and performs secret sharing. However, it is sufficient to have the contribution of a single honest party to ensure the security of the DKG protocol. Leveraging this observation, we can randomly sample a smaller committee, ensuring that the presence of at least one honest party within the committee is guaranteed with a very high probability. In the synchronous network setting, where messages from all honest parties are guaranteed to arrive within a pre-defined time, a significantly smaller committee of dealers can be elected to ensure the presence of at least one honest party [43]. However, in a non-synchronous (e.g., partially synchronous or asynchronous) setting, where up to f_c honest parties may experience arbitrary

delays, additional considerations are required. Specifically, to tolerate up to f_c Byzantine failures, we require that $n_c - f_c \geq f_c + 1$, implying that $n_c \geq 2f_c + 1$ parties are necessary. This condition ensures an honest majority in the clan. Therefore, we select a clan of n_c dealers, consisting of at least $2f_c + 1$ parties, to guarantee an honest majority.

The effectiveness of random sampling-based optimization hinges on the system having lower fault tolerance, enabling the selection of a smaller committee with an honest majority, as determined by the hypergeometric probability distribution. Such techniques are particularly suitable for applications like on-chain randomness [64] or consensus mechanisms, where a super-honest majority assumption is typically upheld.

By randomly selecting committee members, significantly smaller committees can be formed while maintaining the honest majority assumption with negligible failure probability. For instance, in a system with $n = 500$ parties and $f = 166$ faulty parties, a committee of only $n_c = 184$ members suffices to ensure an honest majority, with a negligible failure probability of 10^{-9} , as calculated using the hypergeometric probability distribution. Figure 1 illustrates the committee sizes for various system configurations.

In addition to this honest-majority clan, we randomly sample a much smaller “family” of n_a parties, ensuring at least one honest party in the committee with high probability. This family is responsible for posting information to the blockchain. By using such a reduced committee, we minimize interaction with the blockchain while still guaranteeing that at least one honest party posts the necessary information. This approach effectively balances blockchain interaction efficiency with security, minimizing overhead.

We assume the availability of random beacons to enable the fair and unbiased selection of sub-committees. Random beacons [37], which provide publicly verifiable and unbiased randomness, are widely available on most blockchains [63, 65, 74] at no additional cost. Leveraging their utility, as demonstrated in prior works [43], we use random beacons to ensure fairness and unpredictability in the sub-committee sampling process.

Communication efficient VSS. We build on the VSS protocol of Das et al., which is computationally efficient by requiring each party to verify at most f encrypted shares. In that scheme, a dealer distributes polynomial shares, collects a quorum of signatures on a public commitment, and uses verifiable encryption and reliable broadcast (RBC) to handle missing shares. However, transmitting linear-sized commitments and invoking RBC leads to high communication cost, especially when many dealers are involved.

To reduce communication, we propose a modified VSS. The dealer sends only individual secret shares to parties, without broadcasting commitments. Each party locally computes and signs a commitment to its share and returns the signature. After collecting a quorum of signed commitments, the dealer reconstructs the full commitment vector and generates verifiable encryptions only for parties that did not respond. Crucially, the dealer sends the commitment vector, signatures, and encryptions only to a small elected clan via point-to-point channels, avoiding RBC. Any equivocation by Byzantine dealers is later resolved using the blockchain’s non-equivocation guarantees. This design preserves low computation while substantially reducing communication overhead.

DKG over blockchain using a single consensus instance. To reduce the overhead caused by multiple consensus invocations at various stages of the protocol, we limit the number of consensus instances to a single invocation throughout the protocol execution. Specifically, we send messages via point-to-point channels during the secret sharing phase, avoiding the use of even weaker primitives like RBC. While this approach allows a Byzantine dealer to equivocate and send inconsistent secret shares to different parties, we rely on the non-equivocation guarantees provided by the blockchain to address such inconsistencies at a later point when needed. Additionally, we leverage the blockchain’s built-in consensus capabilities to solve the ACS problem. In doing so, our protocol ensures that sufficient information is recorded on-chain to facilitate the derivation of the final DKG public key directly on the blockchain. This approach achieves the goal of using the blockchain’s consensus mechanism only once, effectively reducing the round complexity of DKG over blockchain and improving overall efficiency.

We note that the secret sharing phase in our protocol is executed entirely off-chain, with information being posted on the blockchain solely to address the ACS problem. This design choice ensures that our framework remains fully asynchronous, allowing it to operate independently of the network’s timing assumptions. Consequently, the exact network model of our protocol is determined by the specific implementation and properties of the underlying blockchain. This flexibility makes our framework adaptable to various blockchains. Notably, this approach enables solving the DKG problem over blockchain in the honest majority setting, even under asynchronous network conditions, as long as blockchain access is available.

Remark. We do not apply committee sampling to scale the consensus protocol itself. A substantial body of work [2, 14, 25, 49] explores this approach, but such protocols require sub-committees with an honest supermajority to run consensus, resulting in large committee sizes unless the system includes over 10,000 participants. Consequently, these techniques are beneficial primarily at very large scales and often provide sub-optimal resilience. Instead, we restrict sub-sampling to selecting the clan of dealers and family for reducing on-chain activity, while treating the blockchain as a black box. This enables performance gains even at moderate system sizes.

Organization. The rest of the paper is organized as follows. The system model and preliminaries are presented in Section 3. We present communication efficient message propagation functions in Section 4. Our proposed protocol is detailed in Section 5, with its evaluation in Section 6. We review related works in Section 7.

3 PRELIMINARIES

We consider a system with n parties $\mathcal{P} := (P_1, \dots, P_n)$, where up to f parties may be Byzantine. The corruption model is static, meaning the adversary determines which parties are corrupted before the protocol starts. Byzantine parties may behave arbitrarily, while non-faulty parties, referred to as *honest*, strictly adhere to the protocol’s specifications.

Our DKG protocol leverages blockchain as a black-box primitive that ensures safety (i.e., parties will not decide on conflicting values) and liveness (i.e., parties will eventually output their input values).

The DKG primitive we design is asynchronous, meaning that messages can take an arbitrarily long time to be delivered but are never lost. Consequently, our protocol is compatible with both synchronous and non-synchronous blockchain protocols, without imposing additional network assumptions on the underlying blockchain.

We employ digital signatures and a public-key infrastructure (PKI) to safeguard against spoofing, replay attacks, and to ensure message authenticity. Each party P_i is associated with a public-private key pair (pk_i, sk_i) . A message x digitally signed by party P_i using its private key is denoted as $\langle x \rangle_i$, while $\langle x \rangle$ refers to an unsigned message x transmitted over an authenticated channel. Additionally, we represent the hash of an input x by $H(x)$, where H is the hash function.

Setup. Let q be a prime number that is $\text{poly}(\kappa)$ bits long, and let $\mathbb{G}$ be a group of order q such that it is computationally infeasible computing the discrete logarithm in $\mathbb{G}$, except with negligible probability in κ . Denote its scalar field by $\mathbb{Z}_q$. Also, let g denote the generator of $\mathbb{G}$.

Tribe, clan and family. When implementing DKG for the system where super-honest majority assumption is required, the entire system of n parties is referred to as the *tribe*, in which up to $f < \frac{n}{3}$ parties may exhibit Byzantine behavior. A sub-committee within this system, where the honest majority assumption is satisfied, is called a *clan*. In a clan, the number of parties is denoted by n_c , with at most $f_c < \frac{n_c}{2}$ being Byzantine, except with negligible probability. Additionally, a *family* represents a much smaller sub-committee of size n_a that contains at least one honest party i.e., $f_a \leq n_a - 1$ may exhibit Byzantine behavior, except with negligible probability.

3.1 Primitives

Linear erasure and error correcting codes. We utilize standard (n, b) Reed-Solomon (RS) codes [72], which encode b data symbols into n -symbol codewords using the Encode function and decode the codewords to recover the original b data symbols using the Decode function. Additional details on Encode and Decode are provided in Appendix A.1.

Cryptographic accumulators. A cryptographic accumulator scheme enables the construction of an accumulation value for a set of values via the Eval function and provides a witness for each individual value in the set using the CreateWit function. Using the accumulation value and a witness, any party can verify whether a specific value is indeed part of the set with the Verify function. More details on these functions are provided in Appendix A.2.

In this paper, we employ *collision-resistant bilinear accumulators* from Nguyen [68], which provide constant-sized witnesses but rely on the q -SDH assumption [16]. Alternatively, Merkle trees [66] can be used to avoid the q -SDH assumption, with an added communication cost of $O(\log n)$.

Verifiable encryption of committed messages. We use verifiable encryption of committed messages, defined as follows:

DEFINITION 1 (VERIFIABLE ENCRYPTION OF A COMMITTED MESSAGE [31]). *Verifiable encryption (VE) of a committed message involves three parties: a prover $\mathcal{Q}$, a verifier $\mathcal{V}$, and a receiver $\mathcal{R}$. The receiver $\mathcal{R}$ holds an encryption-decryption key pair (ek, dk) . Let Cm denote a commitment scheme. Given a tuple (v, c, ek) , the prover $\mathcal{Q}$*

aims to convince the verifier $\mathcal{V}$ that (i) c is a public-key encryption of some message s under the encryption key ek , and (ii) v is a commitment to the same message s . Moreover, the prover demonstrates knowledge of s such that both the commitment and the encryption are consistent with it. A verifiable encryption scheme provides the following interfaces:

- $VE.pp \leftarrow VE.Setup(1^\kappa, aux)$: Generates the scheme parameters $VE.pp$. $VE.pp$ is an implicit input to all other algorithms.
- $(ek, dk) \leftarrow VE.KGen()$: Generate encryption-decryption key pair (ek, dk) used for share encryption and decryption.
- $(c, \pi_{VE}) \leftarrow VE.EncProve(ek, s, v, w)$: The algorithm takes as input a message s and its commitment v with witness w where $(v, w) \leftarrow Cm.Commit(s)$, and outputs a ciphertext c of the tuple $(s, \pi = Cm.Open(v, s, w))$ together with a NIZK proof π_{VE} attesting to the correctness and consistency of the encryption with respect to the commitment.
- $\{0, 1\} \leftarrow VE.Verify(ek, v, c, \pi_{VE})$: A deterministic algorithm that, given the ciphertext c , the proof π_{VE} , and the commitment v , verifies the correctness of c with respect to π_{VE} and v .
- $(s, \pi) \leftarrow VE.Dec(dk, c)$: Given the ciphertext c and the decryption key dk , the algorithm outputs the plaintext obtained by decrypting c under dk .

In this work, we employ the non-interactive verifiable encryption (VE) scheme of Kate et al. [59], which satisfies IND-CPA security, and whose associated proof system guarantees soundness and zero-knowledge.

Batch interfaces. We consider a batch interface to encrypt a batch of messages, as well as a batch interface to verify a vector of encryptions along with their respective proofs of correct sharing. We also consider an interface for aggregating a set of commitments and corresponding encryption vectors into a single aggregated commitment and encryption vector. Such aggregation functionality is supported by several verifiable encryption schemes [53, 59].

- $(\vec{E}, \pi_{VE}) \leftarrow VE.BatchEncProve(I, s, cmt_I, ek_I, wit_I)$: On input a set of indices I , a vector of messages s , their corresponding commitments cmt_I , and witness wit_I , the algorithm outputs a vector of encryptions $\vec{E}$ for each $s \in s$ along with a NIZK proof π_{VE} that satisfies the verification requirements of $VE.BatchVerify$.
- $\{0, 1\} \leftarrow VE.BatchVerify(I, cmt_I, ek_I, \vec{E}_I, \pi_{VE})$: The algorithm outputs 1 if π_{VE} is a valid proof certifying that, for each $i \in I$, there exists (α_i, π_i) such that $(\alpha_i, \pi_i) = VE.Dec(sk_i, c_i)$ and $Cm.Verify(\alpha_i, cmt_i, \pi_i) = 1$.
- $(cmt, \vec{E}) \leftarrow VE.Aggregate(\mathbf{D})$: A deterministic algorithm that takes as input a set $\mathbf{D}$ of tuples of commitments and encryption vectors, $\{(cmt_i, \vec{E}_i)\}$, and outputs the aggregated pair $(cmt, \vec{E})$.

For simulation, we require a zero-knowledge simulation algorithm $VE.BatchSimProve$ that, given an index set I , a message vector s (with some entries possibly zero), and the corresponding commitments cmt_I , outputs simulated encryptions $\vec{E}$ and a simulated proof π_{VE} . The verification algorithm $VE.BatchVerify$ accepts these simulated outputs. By the ZK property, a corrupt verifier cannot distinguish the simulator's output from that of the real algorithm $VE.BatchEncProve$, even when the adversary chooses s . We instantiate this using the class-group-based verifiable encryption of Kate et al. [59], which provides zero knowledge for batched verification in the programmable random oracle model.

Let $\mathcal{B}$ denote an arbitrary set of up to f parties, controlled by the ideal-world adversary $\mathcal{S}$, with $\mathcal{B} \subset [n]$. Let $\ell \in [f, n - f - 1]$ represent the reconstruction threshold. Additionally, let $\mathbb{G}$ be an elliptic curve group of order q with generator g and scalar field $\mathbb{Z}_q$. Initialize an empty set $\mathcal{V} = \emptyset$.

- (1) Upon receiving (Gen, sid, P_i) from an honest party P_i , add i to $\mathcal{V}$ and forward (Gen, sid, P_i) to $\mathcal{S}$. Wait until $|\mathcal{V}| > 1$.
- (2) Sample a random polynomial $F(\cdot)$ of degree ℓ . Set $y = g^{F(0)}$ and $y_i = g^{X_i}$ where $X_i = F(i) \forall i \in [n]$.
- (3) Send $(KEYS, sid, \{y_i\}_{i \in [n]}, y, \{x_i\}_{i \in \mathcal{B}})$ to $\mathcal{S}$.
- (4) Upon receiving $(BIAS, sid, p'(\cdot))$ from $\mathcal{S}$, where $p'(\cdot)$ is a polynomial of degree ℓ , update $y' = y \cdot g^{p'(0)}$, $y'_i = y_i \cdot g^{p'(i)}$ and $X'_i = X_i + p'(i) \forall i \in [n]$.
- (5) Send $(KEYS, X'_i, \{y'_i\}_{i \in [n]}, y')$ to party $P_i \forall i \in [n]$.

Figure 2: Ideal Functionality $\mathcal{F}_{BDKG}$ capturing low threshold and high threshold DKGs with biasable public keys

State machine replication aka blockchain. We leverage existing state machine replication (SMR) protocols or blockchain as a black-box primitive to realize our DKG protocol. The blockchain protocol, executed by a group of parties, processes input client requests and outputs a linearizable log, ensuring a consistent view of the log, comparable to that of a single non-faulty party. It offers the following guarantees, with a formal definition provided for the partially synchronous model in Figure 9 (in Appendix A).

- **Safety.** Honest parties do not commit different values at the same log position.
- **Liveness.** Each client request is eventually committed by all honest parties.

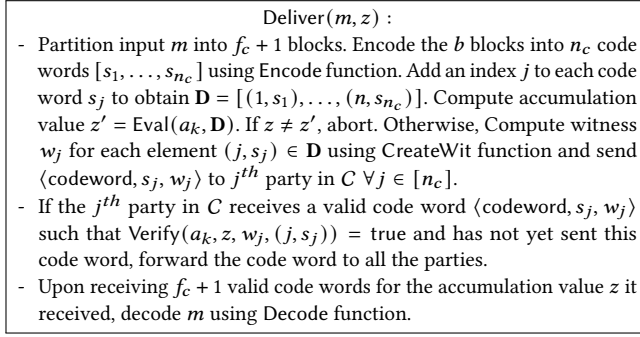
3.2 DKG Definition

A Distributed Key Generation (DKG) protocol for n parties $(P_1, \dots, P_n)$ produces private outputs, referred to as *shares*, denoted by $(X_1, \dots, X_n)$, which collectively represent the shares of a random value $X \in \mathbb{Z}_q$. The protocol also outputs a public value $y = g^X$, where g is a randomly chosen generator of a group $\mathbb{G}$ of order q . Furthermore, each party obtains the threshold public keys corresponding to all parties, specifically g^{X_i} for all $i \in [n]$.

In a DKG protocol, the degree ℓ of the polynomial used for secret sharing is called the *reconstruction threshold*. This threshold specifies the minimum number of parties needed to reconstruct the secret, namely $\ell + 1$, and can range between f and $n - f - 1$. Protocols with $\ell = f$ are referred to as *low-threshold*, while those with $\ell > f$ are referred to as *high-threshold*.

In this work, we focus on a biasable DKG protocol where the adversary is capable of biasing the final public key y . The formal functionality is outlined in Figure 2. Our functionality extends the biasable DKG functionality initially proposed by Cascudo et al. [22] for the synchronous network setting. We adapt their framework to operate in non-synchronous settings and further extend it to support a dual-threshold DKG configuration, wherein the reconstruction threshold ℓ can exceed the corruption threshold f .

A key feature of our functionality is its allowance for the adversary to bias not only the final public key but also the individual threshold public keys of a specified number of parties. In contrast, the DKG functionality of [59] permits biasing the final public key but not the individual public keys in their “Strong mode”, which assumes a majority of honest parties. However, their non-interactive DKG protocol enables a rushing adversary to bias individual public keys by observing the PVSS vectors submitted by honest parties

**Figure 3: Deliver function**

and subsequently selecting their own PVSS vectors accordingly. This adversarial capability is not accurately reflected in their DKG functionality.

Similarly, the biasable DKG functionality (Fig. 8 in [60]) proposed by Katz captures the adversary's ability to bias the final public key but fails to account for the possibility of biasing the individual threshold public keys of participating parties. Our work addresses these limitations, providing a more comprehensive formalization of the adversary's capabilities in biasable DKG protocols.

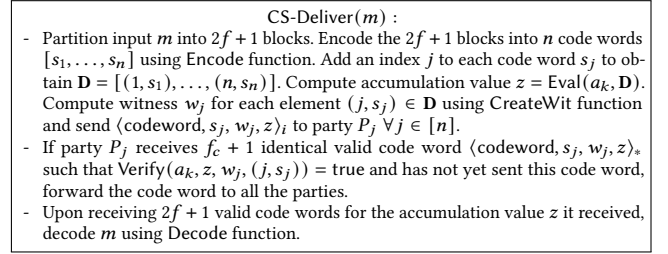
Normalizing the length of cryptographic building blocks. Let λ denote the security parameter, $\kappa_h = \kappa_h(\lambda)$ denote the hash size, $\kappa_a = \kappa_a(\lambda)$ denote the size of the accumulation value and witness of the accumulator and $\kappa_v = \kappa_v(\lambda)$ denote the size of secret share and witness of a secret. Further, let $\kappa = \max(\kappa_h, \kappa_a, \kappa_v)$; we assume $\kappa = \Theta(\kappa_h) = \Theta(\kappa_v) = \Theta(\kappa_a) = \Theta(\lambda)$. Throughout the paper, we can use the same parameter κ to denote the hash size, signature size, accumulator size and secret share size for convenience.

4 EFFICIENT MESSAGE PROPAGATION

This section introduces two efficient propagation primitives—Deliver and CS-Deliver—for disseminating long messages with low communication overhead.

Deliver function. We introduce a Deliver function (see Figure 3) that enables efficient dissemination of long messages within a designated clan C using erasure coding and cryptographic accumulators. The clan of n_c parties is assumed to be selected prior to invocation. Our construction is adapted from RandPiper [13], extended to the asynchronous setting and optimized for efficient intra-clan message propagation.

The Deliver function takes as input a long message m and its corresponding accumulation value z , which represents m . It is assumed that all parties in the clan or tribe have already agreed on the accumulation value z prior to invoking the function. The function operates by encoding the message m into n_c code words $[s_1, \dots, s_{n_c}]$ using $(n_c, f_c + 1)$ RS codes and generating corresponding witnesses $[w_1, \dots, w_{n_c}]$ for each code word. Each code word and its witness pair (s_j, w_j) is then sent to the j^{th} party in the clan, with these pairs being transmitted unsigned. Since all parties have already agreed on the accumulation value z , they can independently verify the validity of each code word using its corresponding witness. This verification is robust against attempts by Byzantine parties to send

**Figure 4: Consortium Sender Deliver function**

multiple or invalid code words, eliminating the need for signing these messages.

Upon receiving the pair (s_j, w_j) for an accumulation value z , the j^{th} party verifies that (j, s_j) corresponds to the j^{th} element of the set $\mathbf{D}$ with the given accumulation value z and the provided witness w_j . Once verified, the j^{th} party forwards the first valid code word and witness it received to other members of the clan. When an honest party collects $f_c + 1$ valid code words corresponding to the accumulation value z , it can decode the original message m .

Invoking the Deliver function on a message of size L bits incurs a communication cost of $O(n_c L + (\kappa + w)n_c^2)$, where κ represents the size of the accumulator and w is the size of the accumulator witness. When using bilinear accumulators, the size of the witness is $O(\kappa)$, while with Merkle trees, it is $O(\kappa \log n_c)$. Consequently, the total communication complexity to propagate a single message of size L is $O(n_c L + \kappa n_c^2)$ bits when relying on bilinear accumulators, or $O(n_c L + \kappa n_c^2 \log n_c)$ bits when using Merkle trees.

Consortium sender deliver function. We also introduce the consortium sender Deliver function (refer to Figure 4), designed to enable a consortium of parties within the clan to efficiently propagate a common long message to all parties in the tribe. This function leverages erasure coding and cryptographic accumulators to optimize message dissemination. Inspired by Dragon-DKG [42], it allows the parties in the clan to collectively and efficiently disseminate a pre-agreed message m to the parties in the tribe.

The CS-Deliver function is invoked by the parties in the clan with a common long message m as input. It encodes the message m into n code words $[s_1, \dots, s_n]$ using $(n, 2f + 1)$ Reed-Solomon (RS) codes and generates corresponding witnesses $[w_1, \dots, w_n]$ for each code word. Each code word and its witness pair (s_j, w_j) is then transmitted to the respective party P_j along with the accumulation value z . Unlike the Deliver function, the message containing the code word, witness, and accumulation value is signed to ensure authenticity.

Each party P_j then awaits receipt of identical valid codeword, witness, and accumulation value (s_j, w_j, z) from at least $f_c + 1$ distinct parties. This ensures that the message m corresponding to the accumulation value z is indeed the value agreed upon by the parties in the clan. Once this condition is satisfied, P_j forwards the tuple (s_j, w_j, z) only once to all other parties in the tribe. Upon collecting at least $2f + 1$ valid codewords associated with the accumulation value z , any honest party can decode and reconstruct the original message.

Invoking the CS-Deliver function on a message of size L bits incurs a communication cost of $O(nL + (\kappa + w)n^2)$, where κ represents

the size of the accumulator and w is the size of the accumulator witness. When using bilinear accumulators, the size of the witness is $O(\kappa)$, while with Merkle trees, it is $O(\kappa \log n_c)$. Consequently, the total communication complexity to propagate a single message of size L is $O(nL + \kappa n^2)$ bits when relying on bilinear accumulators, or $O(nL + \kappa n^2 \log n)$ bits when using Merkle trees.

5 EFFICIENT DKG FOR USE OVER BLOCKCHAIN

In this section, we present an efficient DKG protocol designed for blockchain settings. We first describe the protocol under the super-honest majority assumption, and later extend it to the more relaxed honest-majority case.

Protocol details. We present our efficient DKG protocol in Figure 5. While the protocol is designed to support any threshold within the range $f \leq \ell \leq 2f$, we specifically focus on the highest threshold, $\ell = 2f$, as it is particularly well-suited for applications such as consensus protocols and on-chain randomness. As previously discussed, we assume the parties have access to a randomness beacon RB to select a smaller clan of dealers. The beacon produces an output (run on a predefined timestamp) and a verifiable proof along with it that attests to the correct output computation. The output is used to select the smaller clan of dealers. We rely on the unpredictability (refer to Definition 2) of the RB output to ensure that the clan members are randomly selected.

It is important to note that the contribution of a single honest party is sufficient to ensure the security of the DKG. In a partially synchronous or asynchronous network, $n_c \geq 2f_c + 1$ parties are necessary to ensure this property (as discussed before). Therefore, we select a clan of n_c dealers (denoted by C), consisting of at least $2f_c + 1$ parties, to ensure an honest majority within the clan with high probability. We also elect a family $\mathcal{F}$ consisting of n_a parties, ensuring it includes at least one honest party with high probability. Note that the family $\mathcal{F}$ is significantly smaller in size. To minimize interaction with the blockchain, our protocol allows only the parties in $\mathcal{F}$ to post information onto the blockchain.

To reduce communication overhead, the responsibility of sharing secrets is delegated solely to the clan of dealers. For sharing secrets, we adopt the recent VSS protocol proposed by Das et al. [31], which is known for its computational efficiency. In their scheme, each dealer P_i selects a random degree $2f$ polynomial $p_i(x)$ and sends individual secret shares $p_i(j)$ to each party P_j , along with a linear-sized discrete log commitment cmt_i which are commitment to the coefficients of the secret-sharing polynomial $p_i(x)$. Each party then verifies the correctness of the commitment via SCRAPE's low degree check [21] and checks its received secret share against the commitment cmt_i . If the verification succeeds, the party signs cmt_i and sends the signature back to the dealer. Once the dealer collects a quorum (i.e., a set of $2f + 1$) of valid signatures, denoted as σ_i , the dealers use verifiable encryptions to distribute the secret shares for parties with missing signatures, accompanied by σ_i via a reliable broadcast (RBC) primitive. The RBC primitive is used to ensure the dealer sends consistent value to all the parties. Once the RBC terminates, the parties accept the dealer's secret sharing if σ_i contains a quorum of valid signatures and the encrypted secret shares for the missing parties are valid. If either condition is not

satisfied, the dealer's secret sharing is rejected. Notably, in their scheme, each party is required to verify only f encryptions, which is critical for reducing computational overhead.

While the above secret sharing approach minimizes the computational overhead, it still incurs higher communication when sending the linear sized commitment vector to all the parties which incurs at least $O(\kappa n^2)$ communication per dealer. Moreover, each RBC invocation requires $\Omega(n^2)$ communication [36]. As our DKG protocol involves at least n_c dealers, their approach would incur at least $O(\kappa n_c n^2)$ in communication.

To reduce communication overhead, we propose an improved VSS protocol based on Das et al. [31], which reduces communication in their approach. In our protocol (Step 1 to Step 3), the dealer P_i sends only the individual secret share $p_i(j)$ directly to each party P_j without transmitting any commitments to verify the secret share. Each party P_j computes the discrete log commitment of their share, $g^{p_i(j)}$ (denoted as $\text{cmt}_{i,j}$), signs it, and sends the signature back to the dealer P_i . The dealer P_i collects a quorum of signed commitments for the individual secret shares (denoted as σ_i) and computes the entire commitment vector cmt_i , which contains commitments to the evaluations of the secret-sharing polynomial $p_i(\cdot)$. For parties missing valid signatures, identified as I_i , the dealer generates verifiable encryptions of the corresponding secret shares, denoted by $\tilde{E}_{I_i}$ and its corresponding proof π_{I_i} .

The dealer then transmits the commitment vector cmt_i , the quorum of signatures σ_i , the verifiable encryptions $\tilde{E}_{I_i}$ and its corresponding proof π_{I_i} only to the parties in C via point-to-point channels, avoiding the use of reliable broadcast (RBC). It is worth noting that an honest sender always sends consistent value to all parties in the clan, whereas a Byzantine sender may send equivocating values. We address any such equivocation later using the blockchain, which inherently provides non-equivocation guarantees.

In our protocol, only the clan C of parties is responsible for verifying the commitments, signatures, and the verifiable encryptions. This approach avoids the need for all parties to perform these verifications, reducing the overall computational and communication overhead. Upon receiving the commitment vector cmt_j (Step 4), the quorum of signatures σ_j , and the verifiable encryptions $\tilde{E}_{I_j}$, each party P_i in C performs several verification steps. First, it applies SCRAPE's low-degree check to validate the correctness of the commitment. Next, it verifies the signature $\sigma_{j,k}$ associated with the signed commitment $\text{cmt}_{j,k}$. Finally, it checks the encrypted secret shares corresponding to any missing signatures. If all these checks are successful, party P_i signs a Vote message for the first element of cmt_j i.e., $g^{p_j(0)}$ and the accumulation value of $(\text{cmt}_j, \tilde{E}_{I_j})$, denoted as z_j . The signed Vote message is sent only to the parties in the family $\mathcal{F}$.

Receiving $f_c + 1$ Vote messages for $(\text{cmt}_j, \tilde{E}_{I_j})$ ensures that at least one honest party has verified the correctness of cmt_j , σ_j , and $\tilde{E}_{I_j}$. Consequently, parties in $\mathcal{F}$ wait to receive at least $f_c + 1$ votes for $(\text{cmt}_j, \tilde{E}_{I_j})$. Upon receiving $f_c + 1$ Vote messages for the same $\text{cmt}_j[0]$ and the accumulation value z_j (Step 5), party $P_k \in \mathcal{F}$ forms a quorum certificate for $(z_j, \text{cmt}_j[0])$, denoted as $\text{QC}(z_j, \text{cmt}_j[0])$. Party P_k then updates its meta DKG instance MDKG_k by adding z_j , appends $\text{cmt}_j[0]$ to its commitment list CList_k , and includes

- Each party $P_i \in \mathcal{P}$ executes VE.Setup to generate the public parameters VE.pp , which are agreed upon by all parties. Next, each party P_i generates a key pair $(ek_i, dk_i) \leftarrow \text{VE.KGen}()$, with all parties agreeing on each other's encryption keys. Using a random beacon RB , the parties select a clan C of n_c parties and a family $\mathcal{F}$ of n_a parties from the entire tribe. Note that $n_a \ll n_c \ll n$. Additionally, each party $P_k \in \mathcal{F}$ initializes $\text{MDKG}_k \leftarrow []$, $\text{CList}_k \leftarrow []$ and $\text{LQC}_k \leftarrow []$.
- (1) Each party $P_i \in C$ (as a dealer) performs the following:
 - Sample a $2f$ degree random polynomial $p_i(\cdot)$ such that $p_i(0) = s_i$
 - send $\langle \text{Share}, p_i(j) \rangle_i$ to party $P_j \forall j \in \{1, \dots, n\}$
 - (2) Upon receiving $\langle \text{Share}, p_j(i) \rangle_j$ from the dealer P_j ,
 - $\text{cmt}_{j,i} \leftarrow g^{p_j(i)}$
 - Let $\sigma_{j,i} := \text{Sign}(\text{sk}_i, \text{cmt}_{j,i})$
 - send $\sigma_{j,i}$ to dealer P_j
 - (3) Upon receiving a set σ_i of $2f + 1$ valid signatures, each dealer $P_i \in C$ performs the following:
 - $\text{cmt}_i \leftarrow [g^{p_i(0)}, g^{p_i(1)}, g^{p_i(2)}, \dots, g^{p_i(n)}]$
 - Let I_i be indices of parties with missing valid signatures.
 - $(\vec{E}_{I_i}, \pi_{I_i}) \leftarrow \text{VE.BatchEncProve}(I_i, p_i(j)_{j \in I_i}, \text{cmt}_{I_i}, ek_{I_i}, \perp)$ ▷ wit = $\perp$ when using Dlog commitment
 - send $\langle \text{Propose}, \text{cmt}_i, I_i, \sigma_i, \vec{E}_{I_i}, \pi_{I_i} \rangle_i$ to each party $P_j \in C$
 - (4) Upon receiving $\langle \text{Propose}, \text{cmt}_j, I_j, \sigma_j, \vec{E}_{I_j}, \pi_{I_j} \rangle_j$, each party $P_i \in C$ performs the following:
 - Check $\text{PC.DegCheck}(\text{cmt}_j, 2f)$
 - Check if each $\sigma_{j,k} \in \sigma_j$ is valid signature on $\text{cmt}_{j,k}$ and $|\sigma_j| \geq 2f + 1$
 - Check that each I_j includes indices of all parties with missing valid signatures
 - Check if $\text{VE.BatchVerify}(I_j, \text{cmt}_j, ek_{I_j}, \vec{E}_{I_j}, \pi_{I_j})$
 - if** all the check pass **then**
 - send $\langle \text{Vote}, z_j, \text{cmt}_j[0] \rangle_i$ to each party $P_k \in \mathcal{F}$ ▷ z_j is the accumulation value of $(\text{cmt}_j, \vec{E}_{I_j})$
 - (5) For each party $P_k \in \mathcal{F}$, upon receiving $f_c + 1$ distinct votes for $(z_j, \text{cmt}_j[0])$:
 - $\text{MDKG}_k \leftarrow \text{MDKG}_k \parallel (j, z_j)$
 - $\text{CList}_k \leftarrow \text{CList}_k \parallel \text{cmt}_j[0]$
 - $\text{LQC}_k \leftarrow \text{LQC}_k \parallel \text{QC}(z_j, \text{cmt}_j[0])$ ▷ $\text{QC}(z_j, \text{cmt}_j[0])$ denotes $f_c + 1$ votes on $(z_j, \text{cmt}_j[0])$
 - if** $|\text{MDKG}_k| \geq f_c + 1$ **then**
 - compute g^x from all elements in CList_k
 - Multicast $\langle \text{MetaDKG}, g^x, \text{MDKG}_k, \text{CList}_k, \text{LQC}_k \rangle_k$ to party $P_j \in C$.
 - (6) Upon receiving the first $\langle \text{MetaDKG}, g^x, \text{MDKG}_k, \text{CList}_k, \text{LQC}_k \rangle_k$ from party P_k , each party $P_i \in C$ performs the following:
 - Verify that MDKG_k and CList_k contain $f_c + 1$ elements from distinct parties, each $(z_j, \text{cmt}_j[0])$ paired with a valid QC
 - Check if g^x is computed based on CList_k
 - if** the above check pass **then**
 - send $\langle \text{Ack}, (g^x, z_{mk}) \rangle_i$ to party P_k in $\mathcal{F}$ ▷ z_{mk} is the accumulation value of MDKG_k
 - (7) For each party $P_k \in \mathcal{F}$, upon receiving $f_c + 1$ distinct Ack on z_{mk} , send $(\text{Send}, \text{sid}, (g^x, z_{mk}, \text{AC}(g^x, z_{mk})))$ to $\mathcal{F}_{\text{BC}}$.
 - (8) Each party runs $\mathcal{F}_{\text{BC}}$ on (Get, sid) to read the SMR as TO_i^{sid} . Let $(g^x, z_{mk}, \text{AC}(z_{mk}))$ be the first valid output from $\mathcal{F}_{\text{BC}}$. Invoke $\text{Deliver}(\text{MDKG}_k, z_{mk})$ to propagate MDKG_k .
 - (9) For each party $P_i \in C$ when MDKG_k has been delivered, invoke $\text{Deliver}((\text{cmt}_j, \vec{E}_{I_j}), z_j)$ for all $z_j \in \text{MDKG}_k$.
 - (10) Upon receiving all $(\text{cmt}_j, \vec{E}_{I_j})$ such that $z_j \in \text{MDKG}_k$, each party $P_i \in C$ performs to the following:
 - $(\text{cmt}, \vec{E}) \leftarrow \text{VE.Aggregate}(\{(\text{cmt}_j, \vec{E}_{I_j}) \mid z_j \in \text{MDKG}_k\})$
 - $Q \leftarrow [j \mid (j, z_j) \in \text{MDKG}_k]$ and $x_i \leftarrow \text{VE.Dec}(dk_i, \vec{E}[i]) + \sum_{j \in Q} p_j(i)$
 - send $\langle \text{EncShare}, \vec{E}[j] \rangle_i$ to party $P_j \in \mathcal{P} \setminus C$ and invoke $\text{CS-Deliver}((\text{cmt}, Q))$
 - (11) Upon receiving $\langle \text{EncShare}, \vec{E}[i] \rangle_*$ from $f_c + 1$ parties and list Q , each party $P_i \in \mathcal{P} \setminus C$ performs $x_i \leftarrow \text{VE.Dec}(dk_i, \vec{E}[i]) + \sum_{j \in Q} p_j(i)$.

Figure 5: Efficient DKG Π_{DKG} for use over Blockchain

$\text{QC}(z_j, \text{cmt}_j[0])$ in its list of quorum certificates LQC_k . The presence of $f_c + 1$ Vote messages for the same z_j ensures data availability for $(\text{cmt}_j, \vec{E}_{I_j})$, as at least one honest party must have received $(\text{cmt}_j, \vec{E}_{I_j})$, enabling its retrieval when required. Once MDKG_k

(and CList_k) contains $f_c + 1$ elements, it guarantees that at least one honest party's contribution is included in the meta DKG instance MDKG_k with high probability.

In principle, the meta DKG instance MDKG_k , along with CList_k and the aggregated public key g^x (computed from CList_k), could

be directly posted to the blockchain, with the first instance output by the blockchain being used to derive the DKG secret and public keys. However, storing each LQC, which can grow quadratically in size, on the blockchain is highly inefficient. Moreover, limiting the number of postings to the blockchain by parties in the family $\mathcal{F}$ is desirable. To address these challenges, parties in $\mathcal{F}$ first compute the public key g^x from CList_k by aggregating its elements. They then distribute MDKG_k , CList_k , and the computed g^x to all parties in the clan C , along with the necessary quorum certificates (LQCs) for verification. This approach ensures communication and verification efficiency without overloading the blockchain.

Upon receiving MDKG_k , CList_k , and the corresponding g^x along with the LQCs that certify each element of MDKG_k (and CList_k), and verifying that g^x is the aggregate of all elements in CList_k (Step 6), each party P_i in the clan sends an Ack message for (g^x, z_{mk}) to party P_k , where z_{mk} is the accumulation value of the meta DKG. It is important to note that an honest party sends at most one Ack message per party in the family $\mathcal{F}$. When party P_k receives at least $f_c + 1$ Ack on (g^x, z_{mk}) (denoted by $\text{AC}(g^x, z_{mk})$), it posts $(g^x, z_{mk}, \text{AC}(g^x, z_{mk}))$ to the blockchain for agreement.

Let (g^x, z_{mk}) denote the first valid output from the blockchain. Observe that z_{mk} represents the accumulation value of the meta DKG instance MDKG_k created by party $P_k \in \mathcal{F}$. When P_k is honest, all parties in C receive MDKG_k . However, if P_k is Byzantine, it is possible that only a single honest party receives MDKG_k . To efficiently propagate MDKG_k within C , we leverage the Deliver function. As previously noted, since z_{mk} is already known to all parties, the Deliver function enables efficient dissemination of MDKG_k .

Once MDKG_k has been propagated, all honest parties in C obtain the z_j values included in MDKG_k . Importantly, at least one honest party must have received $(\text{cmt}_j, \vec{E}_{I_j})$. We further use the Deliver function to propagate $(\text{cmt}_j, \vec{E}_{I_j})$ efficiently for all $z_j \in \text{MDKG}_k$.

Upon receiving $(\text{cmt}_j, \vec{E}_{I_j})$ for all $z_j \in \text{MDKG}_k$, the parties in C aggregate these inputs to compute the common pair $(\text{cmt}, \vec{E})$. They first identify the list of dealers in MDKG_k , denoted as Q . Here, $\vec{E}$ contains the aggregated encryption vector for the missing encryptions associated with dealers in Q . The honest parties in the tribe would have already received the remaining secret shares directly from these dealers. Each party P_i in C computes its final secret key by combining the secret shares sent by parties in Q with the decryption of $\vec{E}[i]$.

The parties in C propagate (cmt, Q) using the CS-Deliver primitive. This primitive ensures that a pre-agreed message is delivered from the clan to all parties in the tribe. Additionally, the parties in C send the corresponding $\vec{E}[i]$ to each party $P_i \in \mathcal{P}$. Upon receiving Q , each party P_i computes its partial secret key x'_i using the secret shares sent by parties in Q earlier in the protocol. Additionally, upon receiving at least $f_c + 1$ consistent copies of $\vec{E}[i]$, each party P_i decrypts $\vec{E}[i]$ and combines it with x'_i to compute its final secret key x_i . The threshold public keys for all parties are derived directly from the aggregated commitment.

We prove the security of our protocol Π_{DKG} by proving that it implements $\mathcal{F}_{\text{BDKG}}$. We show that by proving theorem. 1.

THEOREM 1. *Assuming VE is a verifiable encryption scheme satisfying IND-CPA security with a proof system that is sound and has*

a ZK simulator, Sign is a EUF-CMA unforgeable signature scheme, Discrete-Log is hard, the parties are connected via pairwise secure channels and the parties have access to a blockchain functionality $\mathcal{F}_{\text{BC}}$, then Π_{DKG} implements $\mathcal{F}_{\text{BDKG}}$.

We refer to Appendix. D for the proof details.

5.1 Efficiency Analysis

Communication complexity. At the start of the protocol, each party $P_i \in C$ (as a dealer) sends $O(\kappa)$ -sized secret share to all the parties in the tribe, resulting in $O(\kappa n_c n)$ bits of communication. In the next round, all parties in the tribe send a $O(\kappa)$ -sized signature back to the dealers in the clan, also incurring $O(\kappa n_c n)$ in communication. Then, each dealer $P_i \in C$ sends $O(\kappa n)$ -sized message containing the list of signatures, commitment, indices, encryptions, and proof of correct sharing to parties in C , incurring $O(\kappa n_c^2 n)$ bits in communication. Subsequently, the parties in the clan send $O(\kappa)$ -sized Vote message and signature for each dealer to the parties in $\mathcal{F}$ resulting in a communication cost of $O(\kappa n_a n_c^2)$.

Next, parties in $\mathcal{F}$ collect Vote messages and form at least $f_c + 1$ QCs. The size of QC depends on the signature scheme. For instance, its size is $O(\kappa + n_c)$ when using BLS multi-signatures and $O(\kappa)$ or $O(\kappa \log n_c)$ when using threshold signatures. Threshold signatures can be instantiated without needing DKG, resulting in a signature of size $O(\kappa \log n_c)$ with a transparent setup [10] or $O(\kappa)$ with a CRS setup [30], though these threshold signatures remain largely theoretical. Consequently, the size of a meta DKG and CList is $O(\kappa n_c)$ and the size of list of QCs is $O(\kappa n_c + n_c^2)$ when using BLS multi-signatures, or $O(\kappa n_c)$ when using threshold signatures. When the parties in $\mathcal{F}$ send their meta DKG and CList along with the list of QCs to all parties in the clan, the communication complexity is $O(\kappa n_a n_c^2 + n_a n_c^2)$ with multi-signatures, or $O(\kappa n_a n_c^2)$ with threshold signatures.

Next, the parties in C send the Ack messages back to the parties in $\mathcal{F}$, resulting in a communication cost of $O(\kappa n_a n_c)$. Finally, the parties in $\mathcal{F}$ submit the cryptographic accumulator of the meta DKG and the corresponding Ack certificate, with a size of $O(\kappa + n_c)$ with multi-signature or $O(\kappa)$ with threshold signature, to the SMR protocol. Since there are n_a parties in $\mathcal{F}$, up to $O(\kappa n_a + n_a n_c)$ or $O(\kappa n_a)$ bits may be submitted to the SMR protocol.

We analyze the communication cost of the consensus process separately, as it is a necessary step for all implementations to submit the final public key onto the blockchain. When the SMR protocol outputs the cryptographic accumulator of MDKG_k , the parties in C propagate MDKG_k using the Deliver primitive. Since the size of the meta DKG is $O(\kappa n_c)$, by Claim 2, this propagation incurs $O(\kappa n_c^2)$ bits of communication. Additionally, the parties in C propagate the encryption vectors and commitments within MDKG_k . Each encryption and commitment is of size $O(\kappa n)$, and propagating n_c encryptions via the Deliver primitive, as per Claim 2, incurs $O(\kappa n_c^2 n)$ bits of communication. Subsequently, the parties in C send the aggregated encryption to all parties in the tribe, incurring $O(\kappa n_c n)$ communication cost. By Claim 4, delivering the aggregated commitment, which is $O(\kappa n)$ in size, to all parties in the tribe incurs an additional $O(\kappa n^2)$. Consequently, the total off-chain communication cost of the protocol (excluding consensus) is $O(\kappa n_c^2 n + n_a n_c^3 + \kappa n^2)$ when

using multi-signatures, and $O(\kappa n_c^2 n + \kappa n^2)$ when using threshold signatures.

The communication cost for consensus is $\Omega(n\ell + n^2)$ for ℓ bit input [36, 46], with existing constructions achieving $O(n\ell + \kappa n^2)$ [75]. Given an SMR input of size $O(\kappa n_a + n_a n_c)$ with multi-signature or $O(\kappa n_a)$ with threshold signatures, the consensus cost becomes $O(n_a n_c n + \kappa n^2)$ with multi-signatures or $O(\kappa n^2)$ with threshold signatures. We stress again that this communication cost is inherent in all applications that aim to use threshold cryptography over a blockchain.

Round complexity. In the best case, when all dealers are honest, all parties in the clan will receive the meta DKG along with the associated encryptions and commitments, allowing the protocol to complete in 8 rounds plus the consensus latency to commit the first valid meta DKG, which requires at least 3 rounds in the good-case. In contrast, if the dealers are Byzantine, parties in the clan will need to first download the meta DKG and then the associated encryptions, which requires an additional 4 rounds.

Computation complexity. Observe that only parties in C are responsible for verifying the encryptions and list of signatures which amounts to $O(n_c n)$ computation per party. The parties in the $\mathcal{F}$ verify only list of QC which incurs $O(n_c^2)$ computation. The rest of the parties perform $O(n_c)$ computation.

5.2 DKG in the Honest Majority Setting

In the above protocol, we focused on DKG for applications like consensus and on-chain randomness, where the DKG setup involves the parties participating in consensus. For such settings, $f < n/3$ is required in partially synchronous or asynchronous network settings [38].

When setting up DKG for an external system with access to a blockchain, we can tolerate up to $f < n/2$ Byzantine faults within the external system. In such scenarios, our protocol from Figure 5 can be adapted for use over the blockchain with the following modifications. Since the system operates under an honest majority assumption, sampling a clan is unnecessary, as it would result in a sub-committee nearly the size of the original system. Instead, a family of parties can be elected using random beacons to minimize on-chain postings. The required adjustments include setting the degree of the secret sharing polynomial to $f + 1$. Additionally, the CS-Deliver function for propagating aggregated commitments and sending individual aggregate encryptions to the parties becomes unnecessary, as all parties are part of the committee.

Efficiency analysis. The protocol incurs communication complexity of $O(\kappa n^3 + n_a n^3)$, on-chain storage of $O(\kappa n_a + n_a n)$, and a good-case round complexity of 10 rounds. With Byzantine dealers, the round complexity reduces by 2 rounds due to the removal of CS-Deliver. In terms of computation, all parties perform $O(n^2)$ operations. While this protocol incurs higher costs compared to the original protocol, it provides increased fault tolerance.

We present detailed security analysis in Appendix C.

6 EVALUATION

We evaluate the performance of our protocol and compare it with the state-of-the-art standalone DKG protocol by Das et al. [29] for

Table 2: Committee sizes considered for our experiments.

Committee	Number of nodes				
Tribe Size (n)	64	96	128	192	256
Clan Size (n_c)	42	64	80	112	132
Family Size (n_a)	14	16	17	18	18

solving the DKG for use over blockchain, using their docker-based open-source implementation [27].

Implementation details. We implemented our protocol in Rust 1.84, comprising approximately 8,422 lines of code. To instantiate the verifiable encryption of committed messages, we employed the non-interactive class-group VSS [59]. Additionally, we used Merkle trees as cryptographic accumulators. Our implementation is flexible and supports any reconstruction threshold $\ell \geq f$.

Our implementation integrates several libraries, including the BICYCL library [17] for class groups, the Miracl library for cryptographic operations such as elliptic curve cryptography and BLS signatures, the ed25519_dalek library [26] for Ed25519 signatures, and the tokio library [79] for network communication.

The secret and secret shares in our implementation are 256 bits in size, providing a security level of 128 bits. Each element in the class group is compressed to occupy 219 bytes [23]. For elliptic curve operations, we utilize the BLS12-381 curve. Each commitment is represented as a curve element, which requires 48 bytes when encoded.

We utilize the Moonshot [35] SMR protocol to implement the blockchain. Moonshot achieves a commit latency of 3 network hops and supports block proposals at each network hop. We used ED25519 signatures for authentication inside the SMR protocol.

Experimental setup. We carried out our evaluations on the Google Cloud Platform (GCP), distributing nodes evenly across eight distinct GCP regions: Tokyo, Montréal, Oregon, Singapore, Northern Virginia, Los Angeles, London, and Iowa. Each node was hosted on an e2-standard-2 instance, equipped with 2 vCPUs, 8 GB of memory, and up to 4 Gbps of network bandwidth. All nodes operated on Ubuntu 20.04. The blockchain is deployed across the same geographic regions, consisting of a committee of 96 nodes, each running on a separate machine.

We evaluate our implementation by considering various committee sizes ranging from 64 to 256 nodes. Table 2 shows the size of the clan and family committees considered in our experiments for each tribe size (n). The clan and family sizes are selected to ensure a failure probability of less than 10^{-9} . For each n we consider a reconstruction threshold of $\ell = 2f$. All reported timings are averages over 10 runs.

Optimization. In Step 3 of our protocol, the dealer waits for only $2f + 1$ valid signatures before creating encryptions for the missing signatures. However, in practice, since the majority of nodes are honest, the dealer can wait for a timeout duration to collect all the signatures. Once the dealer has received all the signatures, it will not need to create encryptions or corresponding proofs for the missing signatures. As a result, the dealer only needs to forward the signatures for all n nodes, and the clan nodes will only need to verify the signatures without the need to verify any encryptions, leading to improved performance. We refer to this execution as the *optimistic* execution. Additionally, we consider the

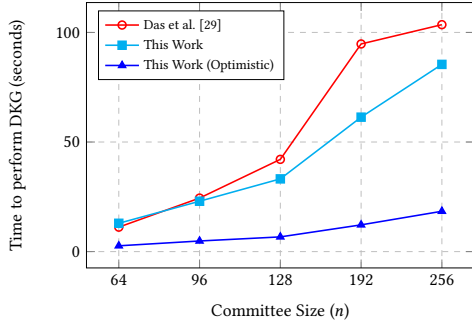


Figure 6: Comparison of time taken to perform DKG for use over blockchain.

regular execution where the dealer waits for $2f + 1$ signatures in Step 3 and sends encryptions for the missing signatures. Both of these cases are evaluated separately in our experiments.

Remark on Das et al. [29] DKG. The DKG protocol by Das et al. [29] is a standalone asynchronous DKG protocol. It requires a common source of randomness (aka common coin) to ensure termination in the asynchronous setting [45]. However, their protocol avoids relying on common coins when Byzantine parties do not actively attack the underlying consensus protocol, allowing it to terminate in a constant number of rounds. This execution is termed *good-case coin-free* execution. In such scenarios, their protocol incurs 18 rounds to terminate. To ensure the final DKG public key appears on the blockchain, an additional 4 rounds are needed when using the Moonshot consensus protocol, resulting in a total of 22 rounds in the good-case. Our evaluation considers only the good-case coin-free execution for their protocol.

6.1 Evaluation Results

Runtime. For our protocol, we measure the time from the protocol’s initiation to the point where $2f + 1$ parties successfully output their secret keys and individual threshold public keys. In contrast, for Das et al. [29], we measure the time from the protocol’s initiation to when a super-majority of parties in the blockchain outputs the public key. The observed average times are illustrated in Figure 6. In the optimistic scenario, no encryptions are generated, leading to significantly improved performance. Our protocol outperforms the ADKG protocol by Das et al. [29] across all system sizes even when encryptions are generated and verified for f parties. This improvement is primarily attributed to our protocol requiring fewer communication rounds. Although the difference in round complexity between our protocol and theirs is significant, the expensive nature of class group encryptions for creation and verification results in a performance improvement that is slightly less than anticipated.

Runtime with parallel processing. To further assess the scalability of our protocol, we conducted additional experiments on GCP e2-highcpu-32 instances, each equipped with 32 vCPU cores, 32 GB of RAM, and 16 Gbps network bandwidth. These specifications are representative of hardware commonly deployed in blockchain networks [9, 78]. By enabling parallel processing, our protocol achieved substantial performance improvements, as illustrated in Figure 7. For instance, with a committee size of 256 nodes, our protocol takes

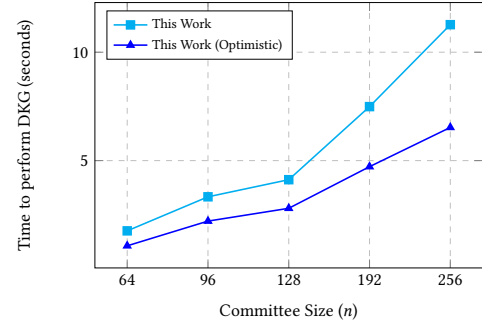


Figure 7: Time taken to perform DKG with parallel processing.

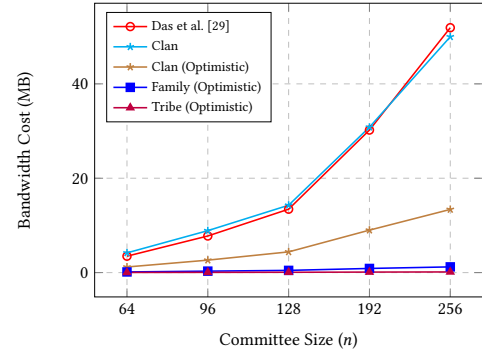


Figure 8: Comparison of bandwidth cost per node.

6.53 seconds in the optimistic scenario and 11.27 seconds in the normal scenario, achieving a speedup of approximately 2.8 times and 7.6 times, respectively, compared to single-threaded execution. These gains stem from the parallelization of computationally intensive tasks, such as generating and verifying class group encryptions and verifying BLS signatures, allowing the protocol to efficiently handle larger system sizes. This enhancement underscores our protocol’s ability to leverage modern multi-core architectures, making it highly suitable for large-scale blockchain deployments.

Bandwidth usage. We also measure the bandwidth usage of both protocols. For our protocol, we evaluate two scenarios: the optimistic scenario and the scenario where the dealers must compute encryptions for f parties. In both cases, the parties outside the clan send minimal data, resulting in significantly lower bandwidth usage. In the optimistic scenario, since the dealer clan does not need to send encryptions or their corresponding proofs, the bandwidth consumption is reduced. However, when encryptions and proofs are required, the bandwidth consumption for clan nodes becomes comparable to the bandwidth overhead per party in Das et al.’s [29] DKG protocol.

Blockchain storage. We compute the gas costs for existing standalone DKG protocols based on the amount of data they must post on-chain to complete the DKG process and have their public key appear on-chain when using the Ethereum blockchain. For this evaluation, we implemented a minimal Solidity contract (compiler version 0.8.26) to post randomized data matching the on-chain requirements of each protocol. Gas costs were measured using the Remix IDE [73] with the Remix VM (Cancun) environment, calculating the total gas required to store this representative data for

Scheme	Committee size (n)				
	64	96	128	192	256
Kate et al. [59]	871.41	1922.68	3387.67	7544.01	13349.01
Das et al. [29]	5.14	7.71	10.28	15.41	20.55
This work	2.59	2.98	3.61	4.24	4.77

Table 3: Comparison of Ethereum gas costs (in $\times 10^6$ gas units) for different committee sizes. Kate et al. [59] require each node to post the complete encrypted dealing and proof on-chain, Das et al. [29] require the nodes to post the final public key on-chain, and our protocol requires only family nodes to post an aggregate public key, an accumulation hash and a BLS multi-signature on-chain.

various committee sizes. In Table 3, we provide associated gas costs for Kate et al. [59], Das et al. [29], and our protocol. It is noteworthy that the gas costs for Cascudo et al. [22] are identical to those of Kate et al. [59]. Similarly, other ADKG protocols would incur gas costs similar to Das et al. [29], as they follow similar mechanisms to publish their public key on-chain.

7 RELATED WORK

An extensive body of work has sought to improve the efficiency of DKG protocols across different network models. However, to the best of our knowledge, existing DKG protocols are designed for the standalone setting and do not ensure that the final DKG public key is recorded on-chain. Moreover, DKG protocols in partially synchronous or asynchronous settings typically tolerate only up to one-third Byzantine faults within the DKG committee. In this work, we present the first discrete-log-based DKG protocol tailored for use over blockchain, leveraging the blockchain’s built-in consensus mechanism and random beacons to achieve efficiency. Notably, our approach also tolerates up to one-half Byzantine faults within the DKG committee. We review the most closely related work below and defer a discussion of synchronous DKG protocols to Appendix B.

Non-interactive VSS and DKG. We extensively compared non-interactive VSS and DKG approaches in the introduction and highlighted their inefficiencies even when provided with access to blockchain and random beacons.

Any-trust DKG [43]. Feng et al. [43] proposed a synchronous DKG protocol in the “any-trust” model, where they select a much smaller committee (similar to our family) with the guarantee that at least one party in the committee is honest, except with negligible probability of error. In the synchronous setting, honest messages are guaranteed to arrive in a timely manner, enabling the selection of a significantly smaller committee. Like our work, they assume random beacons for electing a committee of dealers.

They further assume a synchronous blockchain and use it as a broadcast channel. To optimize on-chain storage, they leverage external storage systems such as IPFS [80], posting only metadata on the blockchain. However, since practical blockchains do not provide synchrony guarantees, their approach becomes impractical in real-world settings. Additionally, their work does not address the final public key appearing on the blockchain.

Partially synchronous and asynchronous DKG protocols. Kate et al. [58] proposed the first practical DKG protocol in the partially synchronous communication model, achieving $O(\kappa n^4)$ communication complexity and $O(n)$ round complexity. Kokoris-Kogias et

al. [61] introduced the first ADKG protocol with optimal resilience, also featuring $O(\kappa n^4)$ communication complexity and $O(n)$ round overhead.

Abraham et al. [4] improved upon this by presenting a protocol with $O(\kappa n^3)$ communication and expected $O(1)$ round complexity. However, their protocol does not support key generation for discrete-log-based cryptosystems and relies on stronger cryptographic assumptions, such as SXDH. Das et al. [32] extended the research by designing a discrete-log-based DKG protocol with $O(\kappa n^3)$ communication and optimal resilience in the asynchronous model, albeit with expected $O(\log n)$ round complexity and requiring the Decisional Composite Residuosity (DCR) assumption. In a follow-up work, Das et al. [29] presented another discrete-log-based DKG protocol with $O(\kappa n^3)$ communication, optimal resilience in the asynchronous model, and a weaker CDH assumption. However, this protocol still incurred an expected $O(\log n)$ round complexity.

More recently, Abraham et al. [3] developed an adaptively secure discrete-log-based DKG protocol with $O(\kappa n^3)$ communication and expected $O(1)$ constant rounds. However, the security of this protocol depends on stronger cryptographic assumptions, including the one-more discrete logarithm (OMDL) assumption and the non-standard algebraic group model (AGM).

Zhang et al. [83] proposed a DKG protocol with minimal assumptions, eliminating the need for PKI and relying solely on the discrete logarithm (DL) assumption. However, their protocol incurs $O(\kappa n^4)$ communication complexity and exhibits exponential runtime in the worst case due to its dependence on local coins for randomness.

As mentioned earlier, all these works focus on the standalone setting, where the final public key is not posted on the blockchain. Furthermore, as discussed in Section 2, even with access to random beacons and blockchain, these approaches cannot achieve the same efficiency as our protocol.

We also note the closely related work on asynchronous proactive secret sharing (APSS) by Günther et al. [52]. Their protocol relies on computationally heavy ACSS primitives such as Haven [8] and single-shot VABA [5], incurring $O(\kappa n^3)$ communication with expected $O(1)$ rounds. They further propose a committee-based variant achieving sub-cubic communication by electing an honest-majority sub-committee, but this requires $O(\log c)$ rounds, where c is the committee size. In contrast, Das et al. [29], which we evaluate against, supports coin-free execution and terminates in constant rounds in the good case. As a result, Günther et al.’s protocol underperforms [29], does not address on-chain storage costs crucial for blockchain deployments, and does not support the simple honest-majority setting.

Concurrent works. Concurrent to our work, Abraham et al. [1] proposed a nearly quadratic communication ADKG protocol with adaptive security, optimal resilience, and expected $O(\kappa)$ round complexity. However, their protocol is not designed to generate keys for discrete-log-based cryptosystems. It also requires stronger setup assumptions, such as the use of verifiable random functions (VRFs), and relies on stronger cryptographic assumptions, including OMDL assumption and AGM.

Similarly, Feng et al. [44] introduced an adaptively secure ADKG protocol with quadratic communication complexity and expected $O(1)$ round complexity. However, their protocol depends on a

stronger setup assumption, specifically a silent setup phase, which incurs cubic communication overhead.

Similarly, Feng et al. [41] propose an asynchronous distributed key reconfiguration protocol that establishes a new DKG for a committee given an existing DKG, with nearly quadratic communication complexity $O(\kappa\lambda n^2)$, where λ is a smaller security parameter. Like our work, they introduce an improved interactive dealing phase based on Das et al. [31] to reduce VSS communication overhead. However, their protocol targets the standalone setting and does not address DKG for use over blockchain.

REFERENCES

- [1] Ittai Abraham, Gilad Asharov, Arpita Patra, and Gilad Stern. Asynchronous agreement on a core set in constant expected time and more efficient asynchronous vss and mpc. In *Theory of Cryptography Conference*, pages 451–482. Springer, 2025.
- [2] Ittai Abraham, TH Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Communication complexity of byzantine agreement, revisited. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, pages 317–326, 2019.
- [3] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, and Gilad Stern. Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In *Annual International Cryptology Conference*, pages 39–70. Springer, 2023.
- [4] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. Reaching consensus for asynchronous distributed key generation. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pages 363–373, 2021.
- [5] Ittai Abraham, Dahlia Malkhi, and Alexander Spiegelman. Asymptotically optimal validated asynchronous byzantine agreement. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, pages 337–346, 2019.
- [6] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. Good-case latency of byzantine broadcast: A complete categorization. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pages 331–341, 2021.
- [7] Nicolas Alhaddad, Mayank Varia, and Ziling Yang. Haven++: Batched and packed dual-threshold asynchronous complete secret sharing with applications. *Cryptology ePrint Archive*, 2024.
- [8] Nicolas Alhaddad, Mayank Varia, and Haibin Zhang. High-threshold avss with optimal communication complexity. In *Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part II 25*, pages 479–498. Springer, 2021.
- [9] Aptos. Node requirements. <https://aptos.dev/en/network/nodes/validator-node/node-requirements>, 2025. Accessed: 2025-03-12.
- [10] Thomas Attema, Ronald Cramer, and Matthieu Rambaud. Compressed σ -protocols for bilinear group arithmetic circuits and application to logarithmic transparent threshold signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 526–556. Springer, 2021.
- [11] Renas Bacho, Christoph Lenzen, Julian Loss, Simon Ochseneither, and Dimitrios Papachristoudis. Grandline: adaptively secure dkg and randomness beacon with (log-) quadratic communication complexity. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 941–955, 2024.
- [12] Carsten Baum, Bernardo David, Rafael Dowsley, Jesper Buus Nielsen, and Sabine Oechsner. TARDIS: A foundation of time-lock puzzles in UC. pages 429–459, 2021.
- [13] Adithya Bhat, Nibesh Shrestha, Zhongtang Luo, Aniket Kate, and Kartik Nayak. Randpiper—reconfiguration-friendly random beacons with quadratic communication. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 3502–3524, 2021.
- [14] Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. Asynchronous byzantine agreement with subquadratic communication. In *Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020, Proceedings, Part I 18*, pages 353–380. Springer, 2020.
- [15] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme. In *International Workshop on Public Key Cryptography*, pages 31–46. Springer, 2002.
- [16] Dan Boneh and Xavier Boyen. Short signatures without random oracles and the sdh assumption in bilinear groups. *Journal of cryptology*, 21(2):149–177, 2008.
- [17] Cyril Bouvier, Guilhem Castagnos, Laurent Imbert, and Fabien Laguillaumie. I want to ride my bicycl: Bicycl implements cryptography in class groups. *Cryptology ePrint Archive*, Paper 2022/1466, 2022. <https://eprint.iacr.org/2022/1466>.
- [18] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.
- [19] Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in constant-time: practical asynchronous byzantine agreement using cryptography. In *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing*, pages 123–132, 2000.
- [20] Ran Canetti, Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Adaptive security for threshold cryptosystems. In *Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*, pages 98–116. Springer, 1999.
- [21] Ignacio Cascudo and Bernardo David. Scrape: Scalable randomness attested by public entities. In *International Conference on Applied Cryptography and Network Security*, pages 537–556. Springer, 2017.
- [22] Ignacio Cascudo and Bernardo David. Publicly verifiable secret sharing over class groups and applications to dkg and yoso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 216–248. Springer, 2024.
- [23] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Two-party ecDSA from hash proof systems and efficient instantiations. In *Annual International Cryptology Conference*, pages 191–221. Springer, 2019.
- [24] Miguel Castro, Barbara Liskov, et al. Practical byzantine fault tolerance. In *OSDI*, volume 99, pages 173–186, 1999.
- [25] Shir Cohen, Idit Keidar, and Alexander Spiegelman. Not a coincidence: Subquadratic asynchronous byzantine agreement whp. In *34th International Symposium on Distributed Computing*, 2020.
- [26] Dalek. Dalek cryptography. <http://dalek.rs>, 2025. Accessed: 2025-01-09.
- [27] Sourav Das. Prototype implementation for the paper practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. <https://github.com/sourav1547/htadkg>.
- [28] Sourav Das, Vinith Krishnan, Irene Miriam Isaac, and Ling Ren. Spurt: Scalable distributed randomness beacon with transparent setup. pages 2502–2517, 2022.
- [29] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In *USENIX Security Symposium*, 2023.
- [30] Sourav Das, Zhuolun Xiang, and Ling Ren. Powers of tau in asynchrony. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society, 2024.
- [31] Sourav Das, Zhuolun Xiang, Alin Tomescu, Alexander Spiegelman, Benny Pinkas, and Ling Ren. Verifiable secret sharing simplified. *Cryptology ePrint Archive*, 2023. Accepted in IEEE SP’25.
- [32] Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous distributed key generation. In *S&P’22*, pages 2518–2534. IEEE, 2022.
- [33] Bernardo David, Bernardo Magri, Christian Matt, Jesper Buus Nielsen, and Daniel Tschudi. GearBox: Optimal-size shard committees by leveraging the safety-liveness dichotomy. pages 683–696, 2022.
- [34] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In *Advances in Cryptology - CRYPTO ’89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 307–315. Springer, 1989.
- [35] Isaac Doidge, Raghavendra Ramesh, Nibesh Shrestha, and Joshua Tobkin. Moonshot: Optimizing chain-based rotating leader bft via optimistic proposals. In *International Conference on Dependable Systems and Networks (DSN)*, 2024.
- [36] Danny Dolev and Rüdiger Reischuk. Bounds on information exchange for byzantine agreement. *Journal of the ACM (JACM)*, 32(1):191–204, 1985.
- [37] Drand. Drand - a distributed randomness beacon daemon. <https://github.com/drand/drand>.
- [38] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)*, 35(2):288–323, 1988.
- [39] Paolo D’Arco and Douglas R Stinson. On unconditionally secure robust distributed key distribution centers. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 346–363. Springer, 2002.
- [40] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*, pages 427–438. IEEE, 1987.
- [41] Hanwen Feng, Yingzi Gao, Yuan Lu, Qiang Tang, and Jing Xu. Practical asynchronous distributed key reconfiguration and its applications. *Cryptology ePrint Archive*, 2025.
- [42] Hanwen Feng, Zhenliang Lu, and Qiang Tang. Dragon: Decentralization at the cost of representation after arbitrary grouping and its applications to sub-cubic dkg and interactive consistency. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*, pages 469–479, 2024.
- [43] Hanwen Feng, Tiancheng Mai, and Qiang Tang. Scalable and adaptively secure any-trust distributed key generation and all-hands checkpointing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 2636–2650. ACM, 2024.

- [44] Hanwen Feng and Qiang Tang. Asymptotically optimal adaptive asynchronous common coin and dkg with silent setup. *Cryptology ePrint Archive*, 2024.
- [45] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM (JACM)*, 32(2):374–382, 1985.
- [46] Matthias Fitzi and Martin Hirt. Optimally efficient multi-valued byzantine agreement. In *Proceedings of the twenty-fifth annual ACM symposium on Principles of distributed computing*, pages 163–168, 2006.
- [47] Pierre-Alain Fouque and Jacques Stern. One round threshold discrete-log key generation without private channels. In *Public Key Cryptography: 4th International Workshop on Practice and Theory in Public Key Cryptosystems, PKC 2001 Cheju Island, Korea, February 13–15, 2001 Proceedings 4*, pages 300–316. Springer, 2001.
- [48] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20:51–83, 2007.
- [49] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th symposium on operating systems principles*, pages 51–68, 2017.
- [50] Jacob Gorman, Lucjan Hanzlik, Aniket Kate, Easwar Vivek Mangipudi, Pratyay Mukherjee, Pratik Sarkar, and Sri Aravinda Krishnan Thyagarajan. Vraas: Verifiable randomness as a service on blockchains. *Cryptology ePrint Archive*, 2024.
- [51] Jens Groth. Non-interactive distributed key generation and key resharing. *Cryptology ePrint Archive*, 2021.
- [52] Christoph U Günther, Sourav Das, and Lefteris Kokoris-Kogias. Practical asynchronous proactive secret sharing and key refresh. *Cryptology ePrint Archive*, 2022.
- [53] Kobi Gurkan, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. Aggregatable distributed key generation. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 147–176. Springer, 2021.
- [54] Timo Hanke, Mahnush Movahedi, and Dominic Williams. Dfinity technology overview series, consensus system. *arXiv preprint arXiv:1805.04548*, 2018.
- [55] Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Cryptographic asynchronous multi-party computation with optimal resilience. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 322–340. Springer, 2005.
- [56] Dennis Hofheinz and Jörn Müller-Quade. A synchronous model for multi-party computation and the incompleteness of oblivious transfer. *Cryptology ePrint Archive*, 2004.
- [57] Aniket Kate and Ian Goldberg. Distributed key generation for the internet. In *2009 29th IEEE International Conference on Distributed Computing Systems*, pages 119–128. IEEE, 2009.
- [58] Aniket Kate, Yizhou Huang, and Ian Goldberg. Distributed key generation in the wild. *LACR Cryptol. ePrint Arch.*, 2012:377, 2012. <https://eprint.iacr.org/2012/377>.
- [59] Aniket Kate, Easwar Vivek Mangipudi, Pratyay Mukherjee, Hamza Saleem, and Sri Aravinda Krishnan Thyagarajan. Non-interactive VSS using class groups and application to DKG. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14–18, 2024*, pages 4286–4300. ACM, 2024.
- [60] Jonathan Katz. Round-optimal, fully secure distributed key generation. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 285–316. Springer, 2024.
- [61] Eleftherios Kokoris Kogias, Dahlia Malkhi, and Alexander Spiegelman. Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1751–1767, 2020.
- [62] Torus Lab. Torus: Globally accessible public key infrastructure for everyone. <https://tor.us/>, 2021.
- [63] Aptos Labs. Roll with move: Secure, instant randomness on aptos. Accessed on May 27, 2025.
- [64] Aptos Labs. Roll with move: Secure, instant randomness on aptos, 2024. Accessed on December 24, 2024.
- [65] Mysten Labs. Unlocking the power of native randomness on sui. Accessed on May 27, 2025.
- [66] Ralph C Merkle. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*, pages 369–378. Springer, 1987.
- [67] Wafa Neji, Kaouthar Blihech, and Narjes Ben Rajeb. Distributed key generation protocol with a new complaint management strategy. *Security and communication networks*, 9(17):4585–4595, 2016.
- [68] Lan Nguyen. Accumulators from bilinear pairings and applications. In Alfred Menezes, editor, *Topics in Cryptology – CT-RSA 2005*, pages 275–292, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [69] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.
- [70] Torben Pryds Pedersen. A threshold cryptosystem without a trusted party. In *Advances in Cryptology—EUROCRYPT’91: Workshop on the Theory and Application of Cryptographic Techniques Brighton, UK, April 8–11, 1991 Proceedings 10*, pages 522–526. Springer, 1991.
- [71] randao.org. Blockchain based verifiable random number generator, 2017. Accessed on Jan 22, 2024.
- [72] Irving S Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the society for industrial and applied mathematics*, 8(2):300–304, 1960.
- [73] Remix. Remix project jump into web3. <https://remix-project.org/>, 2025. Accessed: 2025-01-22.
- [74] Supra Research. On-chain randomness fulfillment via verifiable random functions. Accessed on May 27, 2025.
- [75] Victor Shoup. Sing a song of simplex. In *38th International Symposium on Distributed Computing*, 2024.
- [76] Nibesh Shrestha, Ittai Abraham, Ling Ren, and Kartik Nayak. On the optimality of optimistic responsiveness. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 839–857, 2020.
- [77] Nibesh Shrestha, Adithya Bhat, Aniket Kate, and Kartik Nayak. Synchronous distributed key generation without broadcasts. *IACR Communications in Cryptology*, 1(2), 2024.
- [78] Sui. Sui validator node configuration. <https://docs.sui.io/guides/operator/validator-config>, 2025. Accessed: 2025-03-12.
- [79] Tokio. Tokio: An async runtime for rust. <https://tokio.rs/>, 2025. Accessed: 2025-01-09.
- [80] Dennis Trautwein, Aravindh Raman, Gareth Tyson, Ignacio Castro, Will Scott, Moritz Schubotz, Bela Gipp, and Yiannis Psaras. Design and evaluation of ipfs: a storage layer for the decentralized web. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 739–752, 2022.
- [81] Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Golan Gueta, and Ittai Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, pages 347–356, 2019.
- [82] Thomas Yurek, Licheng Luo, Jaiden Fairbroze, Aniket Kate, and Andrew Miller. hbacss: How to robustly share many secrets. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022*. The Internet Society, 2022.
- [83] Haibin Zhang, Sisi Duan, Chao Liu, Boxin Zhao, Xuanji Meng, Shengli Liu, Yong Yu, Fangguo Zhang, and Liehuang Zhu. Practical asynchronous distributed key generation: Improved efficiency, weaker assumption, and standard model. In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 568–581. IEEE, 2023.

A EXTENDED PRELIMINARIES

A.1 Linear erasure and error correcting codes.

- Encode. Given inputs $m_1, \dots, m_b$, an encoding function Encode computes $(u_1, \dots, u_n) = \text{Encode}(m_1, \dots, m_b)$, where $(u_1, \dots, u_n)$ are code words of length n . A combination of any b elements of the code word uniquely determines the input message and the remaining of the code word.
- Decode. The function Decode computes $(m_1, \dots, m_b) = \text{Decode}(u_1, \dots, u_n)$, and is capable of tolerating up to c errors and d erasures in code words $(s_1, \dots, s_n)$, if and only if $n - b \geq 2c + d$.

A.2 Cryptographic accumulators.

Formally, given a parameter κ , and a set $\mathbf{D}$ of n values $d_1, \dots, d_n$, an accumulator has the following components:

- $\text{Gen}(1^\kappa, n)$: This algorithm takes as input a security parameter κ (represented in unary form 1^κ) and an accumulation threshold n (the maximum number of values that can be securely accumulated), and outputs an accumulator key a_κ . The accumulator key a_κ is part of the q -SDH setup and is publicly accessible to all parties.
- $\text{Eval}(a_\kappa, \mathbf{D})$: This algorithm takes an accumulator key a_κ and a set $\mathbf{D}$ of values to be accumulated, and outputs an accumulation value z for the set $\mathbf{D}$.

- **CreateWit**($a_\kappa, z, d_i, \mathbf{D}$): This algorithm takes an accumulator key a_κ , an accumulation value z for $\mathbf{D}$, and a value d_i ; it returns $\perp$ if $d_i \notin \mathbf{D}$ and a witness w_i if $d_i \in \mathbf{D}$.
- **Verify**(a_κ, z, w_i, d_i): This algorithm takes an accumulator key a_κ , an accumulation value z for $\mathbf{D}$, a witness w_i and a value d_i ; it returns true if w_i is the witness for $d_i \in \mathbf{D}$, and false otherwise. This algorithm takes an accumulator key a_κ , an accumulation value z for $\mathbf{D}$, a witness w_i , and a value d_i ; it returns true if w_i is the correct witness for $d_i \in \mathbf{D}$ and false otherwise.

In this work, we use bilinear accumulator from Nyugen [68] which satisfies the following property:

LEMMA 1 (COLLISION-FREE ACCUMULATOR [68]). *The bilinear accumulator is collision-free. That is, for any set of size n and a probabilistic polynomial-time adversary $\mathcal{A}$, the following function is negligible in κ :*

$$\Pr \left[\begin{array}{l} a_\kappa \leftarrow \text{Gen}(1^\kappa, n), \\ ((d_1, \dots, d_n), d', w') \leftarrow \mathcal{A}(1^\kappa, n, a_\kappa), \\ z \leftarrow \text{Eval}(a_\kappa, \{d_1, \dots, d_n\}) \end{array} \middle| \begin{array}{l} (d' \notin \{d_1, \dots, d_n\}) \wedge \\ (\text{Verify}(a_\kappa, z, w', d') = 1) \end{array} \right]$$

A.3 SCRAPE test [21]

In this work, we employ Feldman commitments to commit to the evaluations of the secret-sharing polynomial. To verify the degree of the commitment efficiently, we utilize the low-degree test introduced in SCRAPE [21]. The protocol employs the PC.DegCheck(cmt) interface, which samples a random degree $n - f - 2$ polynomial $z(\cdot)$ in $\mathbb{F}$ and outputs 1 if the following condition holds:

$$\prod_{i \in [n]} \text{cmt}[i]^{z(i) \cdot \lambda_i} = 1_{\mathbb{G}} \quad (1)$$

where $\lambda_i = \prod_{j \in [n], j \neq i} \frac{1}{(i-j)}$; otherwise outputs 0.

A.4 Blockchain Functionality

We present the simplified ledger/blockchain from Gearbox [33] in Fig. 9, which provides a simple, easy-to-use timed ledger functionality for submitting messages on a global ledger or a blockchain. The functionality $\mathcal{F}_{\text{BC}}$ and the parties have access to a global ticker functionality [12] to keep track of time. The adversary $\mathcal{A}$ can corrupt an arbitrary number of participating parties P_i and run on their behalf. Additionally, $\mathcal{A}$ has the power to order the submitted messages for appending on the ledger. The blockchain functionality guarantees the following security properties:

- (1) **Persistence.** Suppose parties P_i and P_j are honest, and TO_i^{sid} and TO_j^{sid} are outputs of (Get, sid) obtained by P_i and P_j respectively at different times. Then either TO_i^{sid} is a prefix of TO_j^{sid} or vice versa.
- (2) **Bounded timestamps** When a message is submitted to $\mathcal{F}_{\text{BC}}$ at time t (obtained via the ticker functionality [12]), the functionality guarantees that it eventually gets appended to the ledger by the adversary by time $t' \leq t + \Delta$. Additionally, it appears on every honest party P_j 's ledger TO_j^{sid} by time $t' + \Delta$ on issuing a (Get, sid) command by P_j . This property ensures that an adversary can delay appending the message by at most

Δ time where Δ is a bounded delay known to the adversary and the functionality. It also guarantees the liveness of the ledger as all submitted transactions will eventually get appended.

Initialization:
For party P_i set $\text{TO}_i = ()$ and $\text{Sent}^{\text{sid}} = \emptyset$.
Interface for party P_i:
Input: $(\text{Send}, \text{sid}, m)$
(1) Output (P_i, sid, m) to the adversary.
(2) Add (P_i, m, t^{now}) to Sent^{sid} .
Input: (Get, sid)
(1) Output (Get, P_i) to the adversary.
(2) Return TO_i^{sid}
Interface for Adversary:
Input: $(\text{Add}, \text{sid}, m, t)$
(1) Append (m, t) to TO_i^{sid} .
At any time, $\mathcal{F}_{\text{BC}}$ automatically enforces these restrictions:
Persistence: If P_i and P_j are honest, then either TO_i^{sid} is a prefix of TO_j^{sid} or TO_j^{sid} is a prefix of TO_i^{sid} .
Liveness and bounded timestamps: For all messages $(\text{sid}, m, t) \in \text{Sent}^{\text{sid}}$, there is a $t' \leq t + \Delta$ such that by time $t' + \Delta$, we have $(m, t') \in \text{TO}_i^{\text{sid}}$ for all honest P_i .

Figure 9: Simplified Blockchain functionality $\mathcal{F}_{\text{BC}}$ (from Gearbox [33]).

A.5 Randomness Beacon

We recall the formal definition of a randomness beacon from [28] below.

DEFINITION 2 (RANDOMNESS BEACON). *A randomness beacon RB [28] is defined using a tuple of four algorithms:*

- $\text{Gen}(1^\kappa) \rightarrow (\text{vk}, \text{sk})$: Takes as input the security parameter κ and outputs a key pair (vk, sk) , where vk is the public verification key and sk is the secret key.
- $\text{Eval}(\text{sk}, x) \rightarrow (y, \pi)$: Takes as input the secret key sk and input x and outputs a random string $y \in \{0, 1\}^\kappa$ and a proof π attesting to correct computation of y .
- $\text{Verify}(\text{vk}, x, y, \pi) \rightarrow 1/0$: Takes as input the verification key vk , input x , output y , and proof π , and outputs a bit denoting whether the proof verifies or not.

A tuple of algorithms denoted as $\text{RB} = (\text{Gen}, \text{Eval}, \text{Verify})$ is a randomness beacon if it fulfills:

- **Correctness.** For any $\kappa \in \mathbb{N}$, any $(\text{vk}, \text{sk}) \leftarrow \text{Gen}(1^\kappa)$, and any input x , if $(y, \pi) \leftarrow \text{Eval}(\text{sk}, x)$ then $\text{Verify}(\text{vk}, x, y, \pi) \rightarrow 1$.
- **Unpredictability.** For any $\kappa \in \mathbb{N}$, $(\text{vk}, \text{sk}) \leftarrow \text{Gen}(1^\kappa)$, and any set of n valid input-output pairs $\{x_i, y_i, \pi_i\}_{i \in [n]}$, a PPT adversary $\mathcal{A}$ cannot predict the output on an unqueried input x' . More formally, for $\forall n \in \mathbb{N}$, the following holds:

$$\Pr [y' == y'' | (x', y', \pi') \leftarrow \mathcal{A}(\text{vk}, \{x_i, y_i, \pi_i\}_{i \in [n]}) \\ \wedge (y'', \pi'') \leftarrow \text{Eval}(\text{sk}, x')] \leq \text{negli}(\kappa),$$

where (x', y'') has not been computed by RB.

B ADDITIONAL RELATED WORK

Synchronous DKG protocols. Synchronous DKG protocols have been a topic of research for decades [11, 20, 42, 48, 67, 70, 77]. A majority of these protocols assume the availability of a broadcast

channel, which ensures consensus within a predefined time frame. However, practical blockchains operate in a non-synchronous environment, where such strict timing guarantees are not feasible. Consequently, existing blockchains cannot be used to realize the broadcast channel required by these protocols.

Pedersen [70] proposed the first DKG protocol, utilizing a non-interactive information-theoretic verifiable secret sharing scheme. However, Gennaro et al.[48] demonstrated that Pedersen's protocol allows an attacker to bias the public key distribution, leading them to propose a scheme that eliminates this vulnerability, albeit at a higher computational cost. Neji et al.[67] introduced a simpler mechanism to mitigate the bias attack. Canetti et al.[20] extended Gennaro et al.'s work[48], making it secure against adaptive adversaries. Gurkan et al. [53] designed a PVSS-based DKG protocol that produces a public verification transcript of linear size. However, a significant limitation of their protocol is that the secret key is a group element rather than a field element, rendering their approach incompatible with standard threshold signature or encryption schemes widely used in practice.

Recently, several works [11, 42, 77] have proposed DKG protocols designed for point-to-point network settings without assuming broadcast channels. Shrestha et al.[77] achieve $O(\kappa n^3)$ communication complexity with expected $O(1)$ rounds while Feng et al.[42] and Bacho et al.[11] attain sub-cubic communication complexity. However, this comes at the cost of linear round complexity. Furthermore, both Feng et al.[42] and Bacho et al.[11] rely on stronger cryptographic primitives, such as zk-SNARKs, and assume non-standard assumptions like the one-more discrete log assumption (OMDL) to ensure adaptive security[11].

C SECURITY ANALYSIS OF DKG OVER BLOCKCHAIN

Statistical security analysis. When a single clan of n_c parties is randomly selected from a system of n total parties, of which f are Byzantine, the probability of forming a dishonest majority within the clan is given by the hypergeometric cumulative distribution function (CDF). This function calculates the likelihood of selecting more than half of the n_c parties as Byzantine. Specifically, the probability $\Pr(\text{dishonest majority})$ is:

$$\Pr(\text{dishonest majority}) = \sum_{k=\lceil \frac{n_c}{2} \rceil}^{n_c} \frac{\binom{f}{k} \binom{n-f}{n_c-k}}{\binom{n}{n_c}} \quad (2)$$

The value of n_c must be chosen sufficiently large relative to n and f so that the probability of forming a dishonest majority clan is below the desired security threshold μ . Specifically,

$$\Pr(\text{dishonest majority}) \leq 2^{-\mu} \quad (3)$$

Similarly, probability that a randomly selected committee contains no honest party is determined using hypergeometric distribution. Specifically, if there are n parties, of which f are Byzantine, and a committee of size n_a is selected, the probability of selecting a committee with no honest parties is:

$$\Pr(\text{No honest party}) = \frac{\binom{f}{n_a}}{\binom{n}{n_a}} \quad (4)$$

The value of n_a must be selected in proportion to n and f to ensure that the probability of forming a family without any honest party remains below the desired security threshold μ .

$$\Pr(\text{No honest party}) \leq 2^{-\mu} \quad (5)$$

C.1 Correctness Analysis

CLAIM 1. *If an honest party P_i invokes Deliver with a value m corresponding to the accumulation value z , then, except with negligible probability of error, all honest parties will eventually receive the value m .*

PROOF. Suppose an honest party P_i invokes Deliver for a value m associated with the accumulation value z . To do so, P_i must have sent a valid tuple $\langle \text{codeword}, s_j, w_j \rangle$, derived from the value m , to the j^{th} party in C .

Upon receiving the tuple $\langle \text{codeword}, s_j, w_j \rangle$, the j^{th} party forwards its code word to all other parties in C . Consequently, all parties in C will receive at least $f_c + 1$ valid code words, which are sufficient to decode the value m . However, these guarantees rely on C having an honest majority, a condition that fails with only negligible probability of error. $\square$

CLAIM 2 (COMMUNICATION COMPLEXITY OF Deliver). *Let κ denote the size of the accumulator and w represent the size of the witness. Invoking Deliver with an input of size L incurs a communication complexity of $O(n_c \ell + w n_c^2)$.*

PROOF. When a Deliver operation is invoked on an object of size L , each party in the C multicasts a code word of size $O(L/n_c)$, along with a witness of size w . Consequently, the total communication complexity amounts to $O(n_c L + w n_c^2)$. $\square$

CLAIM 3. *When a consortium of at least $f_c + 1$ senders in the C invoke CS-Deliver to propagate the common value m , then all honest parties in $\mathcal{P}$ will eventually receive the value m , except with negligible probability of error.*

PROOF. Suppose a consortium of at least $f_c + 1$ senders P_i invokes CS-Deliver to propagate a value m . Each of these $f_c + 1$ senders must have transmitted a valid tuple $\langle \text{codeword}, s_j, w_j, z \rangle$, derived from the value m , to the party $P_j \in \mathcal{P}$.

As a result, each honest party $P_j \in \mathcal{P}$ must have received a consistent and valid code word s_j from at least $f_c + 1$ parties. Party P_j then forwards the tuple $\langle \text{codeword}, s_j, w_j, z \rangle$ to all honest parties. Consequently, each honest party P_i will eventually receive at least $2f + 1$ valid code words corresponding to the accumulation value z , which are sufficient to decode the value m . $\square$

CLAIM 4 (COMMUNICATION COMPLEXITY OF CS-Deliver). *Let κ denote the size of the accumulator and w represent the size of the witness. When a consortium of senders invokes CS-Deliver on an input of size L , the communication complexity is $O(nL + (\kappa + w)n^2)$.*

PROOF. Each party P_i that invokes CS-Deliver sends a code word of size $O(L/n)$, along with a witness of size w and an accumulator of size κ , to all parties in $\mathcal{P}$. This results in a communication cost of $O(L + (\kappa + w)n)$. For n_c parties, the total communication is $O(n_c L + (\kappa + w)n_c n)$. In the second step, all parties in $\mathcal{P}$ send a code word of size $O(L/n)$, along with a witness of size w and an accumulator of size κ , to all other parties. This results in a communication complexity of $O(nL + (\kappa + w)n^2)$. Thus, the total communication complexity is $O(nL + (\kappa + w)n^2)$. $\square$

CLAIM 5. *If an honest party sends an Ack for a meta-DKG instance MDKG_k , then for each commitment $\text{cmt}_j \in \text{MDKG}_k$, (i) all honest parties have either received valid secret shares or their valid secret shares are encrypted in $\tilde{E}_{I_j}$, and (ii) at least one honest party in C has received the pair $(\text{cmt}_j, \tilde{E}_{I_j})$, except with negligible probability of error.*

PROOF. An honest party P_i sends an Ack for a meta-DKG instance MDKG_k only upon receiving an accompanying LQC, which contains at least $f_c + 1$ signatures for each commitment $\text{cmt}_j \in \text{MDKG}_k$. The presence of $f_c + 1$ signatures for a commitment cmt_j implies that at least one honest party has received σ_j and verified that it includes $2f + 1$ valid signatures on cmt_j and verified encrypted secret shares $\tilde{E}_{I_j}$ for any missing valid signatures. Consequently, this ensures that all honest parties have received either valid secret shares or that their encrypted secret shares are included in $\tilde{E}_{I_j}$ for each $\text{cmt}_j \in \text{MDKG}_k$. Furthermore, it guarantees that at least one honest party in C has received the pair $(\text{cmt}_j, \tilde{E}_{I_j})$. These guarantees, however, do not hold if C has a dishonest majority, which occurs with only negligible probability of error. $\square$

LEMMA 2. *Let $(g^x, z_{mk}, \text{AC}(z_{mk}))$ be the output from the SMR. All honest parties will receive aggregated commitment cmt and final secret key, except with a negligible probability of error.*

PROOF. Note that $\text{AC}(z_{mk})$ consists of an Ack message from at least one honest party in C , with high probability. By Claim 5, all honest parties must have either received the secret shares corresponding to each commitment $\text{cmt}_j \in \text{MDKG}_k$, or their encrypted secret shares are included in $\tilde{E}_{I_j}$. Additionally, at least one honest party in C has received the pair $(\text{cmt}_j, \tilde{E}_{I_j})$. After the SMR outputs $(g^x, z_{mk}, \text{AC}(z_{mk}))$, the honest parties in C invoke the Deliver function to propagate $(\text{cmt}_j, \tilde{E}_{I_j})$ for every $\text{cmt}_j \in \text{MDKG}_k$. Consequently, by Claim 1, all honest parties in C will eventually receive $(\text{cmt}_j, \tilde{E}_{I_j})$ for each $\text{cmt}_j \in \text{MDKG}_k$.

Consequently, each honest party in C aggregates the commitments and missing encryption vectors to compute the aggregated commitment cmt and the aggregated encryption vector for missing shares, $\tilde{E}$. Let Q denote the list of dealers included in MDKG_k . Each party P_i in C then combines the secret shares received from dealers in Q with the decryption of $\tilde{E}[i]$ to derive its final secret key x_i .

Additionally, the parties in C invoke the CS-Deliver function to disseminate (cmt, Q) and distribute the corresponding $\tilde{E}[i]$ to each party $P_i \in \mathcal{P}$. By Claim 3, each party $P_i \in \mathcal{P}$ will receive (cmt, Q) and at least $f_c + 1$ consistent shares of $\tilde{E}[i]$. Subsequently, each party P_i combines the secret shares obtained from dealers in Q with the decrypted values of $\tilde{E}[i]$ to compute its final secret key x_i . These

guarantees hold unless C contains a dishonest majority, an event that occurs with only negligible probability. $\square$

LEMMA 3. *Let x denote the final secret key and x_i the final individual secret key for party P_i . All honest parties agree on the same public key g^x and the corresponding threshold public keys g^{x_i} for all $i \in [n]$, except with a negligible probability of error.*

PROOF. By the agreement property of the underlying SMR or blockchain, all parties output a common value $(g^x, z_{mk}, \text{AC}(z_{mk}))$, ensuring that all honest parties agree on the same public key g^x . Furthermore, by Lemma 2, all honest parties receive the same aggregated commitment cmt . Since these commitments correspond to the evaluations of the secret-sharing polynomial, the aggregated commitment cmt serves as the commitment to the individual secret key of each party. Consequently, the threshold public key for each party can be derived directly from cmt . $\square$

D PROOF OF THEOREM 1

We provide the simulator algorithm in Fig. 10 and 11. The simulator receives the $g^{F(\cdot)}$ values from $\mathcal{F}_{\text{BDKG}}$. To simulate the honest dealers: one random dealer $\hat{P} \in C$ is simulated s.t. its key is set to $g^{F(\cdot)}$ and the others are honestly run with random secrets. If the simulated dealer is not selected in the final set then rewind the simulation until $\hat{P}$ is selected in the final set. And once the simulated dealer $\hat{P}$ is selected then set the polynomial $p'(\cdot) = \tilde{p}(\cdot) + p_H(\cdot)$ where $p_H(\cdot)$ is the sum of polynomial of the honest dealers in the final set, and $\tilde{p}(\cdot)$ is the sum of polynomial of the corrupt dealers selected in the set. $\tilde{p}(\cdot)$ can be extracted as the simulator controls $n - f$ parties among n parties.

We prove that our protocol in Fig. 5 implements $\mathcal{F}_{\text{BDKG}}$ in the real-ideal world paradigm by proving Thm. 1.

PROOF. We assume that there exists a PPT adversarial algorithm $\mathcal{A}$ that corrupts f participating parties in the protocol execution. In the real-world execution of the protocol, $\mathcal{A}$ corrupts the set $\mathcal{B}$ of f parties and interacts with the rest $n - f$ of the honest parties. At the end of the protocol execution, we denote its view as $\text{Real}_{\Pi_{\text{BDKG}}, \mathcal{A}}(1^K)$. In the ideal world, execution of the protocol, the honest parties provide their input to $\mathcal{F}_{\text{BDKG}}$ and we provide a PPT simulator $\mathcal{S}$ that given access to the adversarial algorithm $\mathcal{A}$ and the functionality $\mathcal{F}_{\text{BDKG}}$ produces the ideal world adversary view $\text{Ideal}_{\mathcal{F}_{\text{BDKG}}, \mathcal{S}}(1^K)$. According to the real-ideal world paradigm, these two views should be indistinguishable. We provide a hybrid argument to argue indistinguishability between the real and ideal world distributions.

- Hyb_0 : This is the real-world execution of the protocol.
- Hyb_1 : Same as Hyb_0 , except the simulator simulates the honest parties and aborts the protocol execution if there are more than f_c corrupt parties in the randomly sampled clan C or if there are more than f_a corrupt parties in family $\mathcal{F}$. Indistinguishability follows from the unpredictability of the random beacon at the time of static corruption. Consequently, the adversary's view is statistically indistinguishable except with negligible probability $2^{-\mu}$ (see Section 5.1).
- Hyb_2 : Same as Hyb_1 , except the simulator invokes the NIZK simulation algorithm VE.BatchSimProve of VE to

compute the batched proofs for all the honest parties. Indistinguishability follows from the zero-knowledge property of the VE scheme.

- Hyb_3 : Same as Hyb_2 , except $\mathcal{S}$ decrypts (in Step 4) the encryptions in $\vec{E}_{I_j}$ corresponding to the honest parties in I_j (except when P_j is the simulated node, i.e. $P_j = \widehat{P}$) and checks that they match with the commitment cmt_j . If they don't match then P_j is considered as invalid. Note that we use Discrete log commitments, and so there is no commitment randomness and the commitment opening is the decrypted message. Indistinguishability follows from the binding of the commitment scheme (in fact, it is perfectly binding as Discrete-Log commitments are used), correctness of the encryption in VE, and soundness of the NIZK used in VE.
- Hyb_4 : Same as Hyb_3 , except $\mathcal{S}$ encrypts 0 in $\vec{E}_j$ corresponding to the honest receivers for $j \in I_i \cap \mathcal{H}$. Indistinguishability follows from the IND-CPA security of VE.
- Hyb_5 : Same as Hyb_4 , except $\mathcal{S}$ aborts if the adversary forges signatures of honest parties. Indistinguishability follows from the EUF-CMA property of the signature scheme.
- Hyb_6 : Same as Hyb_5 , except $\mathcal{S}$ samples a random honest party $\widehat{P}$ from C , and it sends 0 on behalf of $\widehat{P}$ to other honest

parties. Indistinguishability follows as the honest parties are connected via secure channels.

- Hyb_7 : Same as Hyb_6 , except $\mathcal{S}$ rewinds to step 0 if $\widehat{P}$ is not selected in the final MDKG_k . It is identical to the previous hybrid and the simulator needs to rewind at most v times where $(\frac{f_c}{f_c+1})^v < 2^{-\mu}$. The final chosen MDKG_k can contain at most f_c parties and one honest party, and this one honest party will not be $\widehat{P}$ with probability $\frac{f_c}{f_c+1}$. Repeating it v times, this error probability becomes statistically negligible and $\widehat{P}$ would be chosen in MDKG_k .
- Hyb_8 : Same as Hyb_7 , except $\mathcal{S}$ obtains $g^{F(\cdot)}$ from $\mathcal{F}_{\text{BDKG}}$ simulates $\widehat{P}$ according to the ideal world simulation, extracts $p(\cdot)$ from the corrupt parties as it controls $n - f$ parties, and then $\mathcal{S}$ invokes the simulator of $\mathcal{F}_{\text{BC}}$ s.t. the MDKG_k containing $g^{F(\cdot)}$ is in the final set, and invokes $\mathcal{F}_{\text{BDKG}}$ with input $p'(\cdot) := p_H(\cdot) + \widetilde{p(\cdot)}$. Indistinguishability follows in the $\mathcal{F}_{\text{BC}}$ model as the simulator of $\mathcal{F}_{\text{BC}}$ has the ability to decide the order (of the submitted messages) that will appear on the ledger. This is the ideal world execution of the protocol.

□

The adversary corrupts a set $\mathcal{B}$ of parties where $|\mathcal{B}| = f$ for corruption threshold f and $n \geq 3f + 1$. The set of honest parties are denoted as $\mathcal{H} = [n] \setminus \mathcal{B}$.

Initialization Step:

- The honest parties $\{P_i\}_{i \in \mathcal{H}}$ and simulator (on behalf of corrupt parties in $\mathcal{B}$) invoke the functionality $\mathcal{F}_{\text{BDKG}}$ (Fig. 2) with input $(\text{Gen}, \text{sid}, P_i)$ with $\ell := 2f$. The functionality does the following: Sample a random polynomial $F(\cdot)$ of degree $\ell = 2f$. Set $y = g^{F(0)}$ and $y_i = g^{F(i)} \forall i \in [n]$. Send $(\text{KEYS}, \text{sid}, \{y_i\}_{i \in [n]}, y, \{X_i\}_{i \in \mathcal{B}})$ to $\mathcal{S}$ where $X_i := F(i), \forall i \in [n]$.
- If there are more than f_c corrupt dealers in the randomly sampled clan C or more than f_a corrupt parties in $\mathcal{F}$, then the Simulator aborts.
- The simulator simulates the generation of $(ek_i, dk_i) \leftarrow \text{VE.KGen}()$ key pairs for each $i \in \mathcal{H}$ by running the honest algorithm.

(0) $\mathcal{S}$ selects a random honest dealer node $\hat{P} \in C$ to plug-in the $\mathcal{F}_{\text{BDKG}}$ instance.

(1) $\mathcal{S}$ simulates the honest dealers nodes $P_i \in C \cap \mathcal{H} \setminus \hat{P}$ by running the honest dealer algorithm:

- Sample a $2f$ degree random polynomial $p_i(x)$ such that $p_i(0) = s_i$.
- send $\langle \text{Share}, p_i(j) \rangle_i$ to party $P_j \forall j \in [n]$.

$\mathcal{S}$ simulates the honest dealer $\hat{P} \in C$ by:

- Setting $p_{\hat{P}}(j) := \{X_j\} = F(j)$ for $j \in \mathcal{B}$ and $p_{\hat{P}}(j) := \{0\}$ for $j \in [n] \setminus \mathcal{B}$.
- $\mathcal{S}$ sends $\langle \text{Share}, p_{\hat{P}}(j) \rangle_{\hat{P}}$ to party $P_j \forall j \in [n]$ on behalf of $\hat{P}$. We denote this node as $\hat{P} := P_{\hat{P}}$.

(2) The simulator receives $p_j(i)$ shares from all dealers $P_j \in C$ on behalf of simulated parties $P_i \in \mathcal{H}$, where $|\mathcal{H}| \geq 2f + 1$.

- The simulator extracts the corrupt dealer polynomials. It interpolates all the adversarially chosen polynomials $p_j(\cdot)$ as the simulator contains $2f + 1$ shares for each $2f$ -degree polynomial $p_j(\cdot)$ for $j \in C \cap \mathcal{B}$. If interpolation fails (can be verified by Reed Solomon error detection), then the simulator sets $p_j(\cdot) = \perp$.

The simulator runs the honest party algorithm (for $P_j \in \mathcal{H} \setminus \hat{P}$) to simulate the honest parties as follows:

$\text{cmt}_{j,i} \leftarrow g^{p_j(i)}$
 Let $\sigma_{j,i} := \text{Sign}(\text{sk}_i, \text{cmt}_{j,i})$
 send $\sigma_{j,i}$ to dealer P_j

To simulate $\hat{P} = P_{\hat{P}}$, the simulator does the following:

$\text{cmt}_{\hat{P},i} \leftarrow y_i$
 Let $\sigma_{\hat{P},i} := \text{Sign}(\text{sk}_i, \text{cmt}_{\hat{P},i})$
 send $\sigma_{\hat{P},i}$ to dealer $\hat{P}$

(3) $\mathcal{S}$ simulates the honest dealer nodes $P_i \in C \cap \mathcal{H} \setminus \hat{P}$ by running the honest dealer algorithm.

- Upon receiving a set σ_i of $2f + 1$ valid signatures, if there are forged signatures (that $\mathcal{S}$ did not generate and it verifies against an honest party's public key pk) then $\mathcal{S}$ aborts, otherwise each simulated dealer $P_i \in C \cap \mathcal{H} \setminus \hat{P}$ performs the following:
 $\text{cmt}_i \leftarrow [g^{p_i(0)}, g^{p_i(1)}, g^{p_i(2)}, \dots, g^{p_i(n)}]$
 Let I_i be indices of parties with missing valid signatures.
 $(\vec{E}_{I_i}, \pi_{I_i}) \leftarrow \text{VE.BatchEncProve}(I_i, p_i(j)_{j \in I_i}, \text{cmt}_{I_i}, \text{ek}_{I_i}, \perp)$
 send $\langle \text{Propose}, \text{cmt}_i, I_i, \sigma_i, \vec{E}_{I_i}, \pi_{I_i} \rangle_i$ to each party $P_j \in C$
- $\mathcal{S}$ simulates $\hat{P} \in C$ as follows.
 $\text{cmt}_{\hat{P}} \leftarrow [y_1, y_2, \dots, y_n]$
 Let $I_{\hat{P}}$ be indices of parties with missing valid signatures.
 Compute the simulated encryptions (to 0) for honest parties, encryptions (to $p_{\hat{P}}(j)$) for corrupt parties and simulated proofs as $(\vec{E}_{I_{\hat{P}}}, \pi_{I_{\hat{P}}}) \leftarrow \text{VE.BatchSimProve}(I_{\hat{P}}, p_{\hat{P}}(j)_{j \in I_{\hat{P}} \cap \mathcal{B}} \parallel \{0\}_{j \in I_{\hat{P}} \cap \mathcal{H}}, \text{cmt}_{I_{\hat{P}}}, \text{ek}_{I_{\hat{P}}})$ where $\{p_{\hat{P}}(j)\}_{j \in I_{\hat{P}} \cap \mathcal{B}} = \{F(j)\}_{j \in I_{\hat{P}} \cap \mathcal{B}} = \{X_j\}_{j \in I_{\hat{P}} \cap \mathcal{B}}$.
 send $\langle \text{Propose}, \text{cmt}_{\hat{P}}, I_{\hat{P}}, \sigma_{\hat{P}}, \vec{E}_{I_{\hat{P}}}, \pi_{I_{\hat{P}}} \rangle_{\hat{P}}$ to each party $P_j \in C$

(4) Upon receiving $\langle \text{Propose}, \text{cmt}_j, I_j, \sigma_j, \vec{E}_{I_j}, \pi_{I_j} \rangle_j$, each simulated party $P_i \in C$ runs the following algorithm:

Check $\text{PC.DegCheck}(\text{cmt}_j, 2f)$
 Check if each $\sigma_{j,k} \in \sigma_i$ is valid signature on $\text{cmt}_{j,k}$ and $|\sigma_j| \geq 2f + 1$
 Check that each I_j includes indices of all parties with missing valid signatures
 If $P_j = \hat{P}$, then skip this check and proceed to next step. Otherwise $\mathcal{S}$ decrypts the encryptions in $\vec{E}_{I_j}$ corresponding to the honest parties in $I_j \setminus$ (using their decryption keys) and checks that they match with the commitment cmt_j . If they don't match then P_j is considered as invalid.
if all the check pass **then**
 send $\langle \text{Vote}, z_j, \text{cmt}_j[0] \rangle_i$ to each party $P_k \in \mathcal{F}$ $\triangleright z_j$ is the accumulation value of $(\text{cmt}_j, \vec{E}_{I_j})$

Figure 10: Simulator for our DKG Protocol

- (5) For each simulated party $P_k \in \mathcal{F}$ runs the honest algorithm: Upon receiving $f_c + 1$ distinct votes for $(z_j, \text{cmt}_j[0])$:
- $\text{MDKG}_k \leftarrow \text{MDKG}_k \parallel (j, z_j)$
 $\text{CList}_k \leftarrow \text{CList}_k \parallel \text{cmt}_j[0]$
 $\text{LQC}_k \leftarrow \text{LQC}_k \parallel \text{QC}(z_j, \text{cmt}_j[0])$ ▷ $\text{QC}(z_j, \text{cmt}_j[0])$ denotes $f_c + 1$ votes on $(z_j, \text{cmt}_j[0])$
if $|\text{MDKG}_k| \geq f_c + 1$ **then**
 compute g^x from all elements in CList_k
 Multicast $\langle \text{MetaDKG}, g^x, \text{MDKG}_k, \text{CList}_k, \text{LQC}_k \rangle_k$ to party $P_j \in \mathcal{C}$.
- (6) Upon receiving the first $\langle \text{MetaDKG}, g^x, \text{MDKG}_k, \text{CList}_k, \text{LQC}_k \rangle_k$ from party P_k , each simulated party $P_i \in \mathcal{C}$ runs the honest algorithm by performing the following:
- Verify that MDKG_k and CList_k contain $f_c + 1$ elements from distinct parties, each $(z_i, \text{cmt}_i[0])$ paired with a valid QC
 Check if g^x is computed based on CList_k
if the above check pass **then**
 send $\langle \text{Ack}, (g^x, z_{mk}) \rangle_i$ to party P_k in $\mathcal{F}$ ▷ z_{mk} is the accumulation value of MDKG_k
- (7) If z_{mk} does not contain the commitment of $\widehat{P}$, i.e. $\text{cmt}_{\widehat{P}}$, then $\mathcal{S}$ rewinds to Step. 0. Otherwise, or each party $P_k \in \mathcal{F}$, upon receiving $f_c + 1$ distinct Ack on z_{mk} , send $(\text{Send}, \text{sid}, (g^x, z_{mk}, \text{AC}(g^x, z_{mk})))$ to $\mathcal{F}_{\text{BC}}$.
- (8) $\mathcal{S}$ invokes the simulator of $\mathcal{F}_{\text{BC}}$ s.t. the first valid output $(g^x, z_{mk}, \text{AC}(g^x, z_{mk}))$ from $\mathcal{F}_{\text{BC}}$ contains $\text{cmt}_{\widehat{P}}$. Each party runs $\mathcal{F}_{\text{BC}}$ on (Get, sid) to read the SMR as TO_i^{sid} .
 Invoke $\text{Deliver}(\text{MDKG}_k, z_{mk})$ to propagate MDKG_k .
- (9) For each simulated party $P_i \in \mathcal{C}$ when MDKG_k has been delivered, invoke $\text{Deliver}((\text{cmt}_j, \vec{E}_{I_j}), z_j)$ for all $z_j \in \text{MDKG}_k$.
- (10) Upon receiving all $(\text{cmt}_j, \vec{E}_{I_j})$ such that $z_j \in \text{MDKG}_k$,
- $(\text{cmt}, \vec{E}) \leftarrow \text{VE.Aggregate}(\{(\text{cmt}_j, \vec{E}_{I_j}) \mid z_j \in \text{MDKG}_k\})$
 $Q \leftarrow [j \mid (j, z_j) \in \text{MDKG}_k \text{ and } x_i \leftarrow \text{VE.Dec}(\text{dk}_i, \vec{E}[i]) + \sum_{j \in Q} p_j(i)]$
 send $\langle \text{EncShare}, \vec{E}[j] \rangle_i$ to party $P_j \in \mathcal{P} \setminus \mathcal{C}$ and invoke $\text{CS-Deliver}((\text{cmt}, Q))$
- (11) Upon receiving $\langle \text{EncShare}, \vec{E}[i] \rangle_*$ from $f_c + 1$ parties and Q list and perform $x_i \leftarrow \text{VE.Dec}(\text{dk}_i, \vec{E}[i]) + \sum_{j \in Q} p_j(i)$. The simulator completes the simulation by invoking $\mathcal{F}_{\text{BDKG}}$ as follows:
- Let us denote the final commitment $\text{cmt} = \text{cmt}_{\widehat{P}} + \text{cmt}_{j \in (\vec{E}_{I_j} \cap \mathcal{H}) \setminus \widehat{P}} + \text{cmt}_{j \in (\vec{E}_{I_j} \cap \mathcal{B}) \setminus \widehat{P}}$
 - Let $p_H(\cdot)$ denote the sum of polynomials of honest parties P_j , except $\widehat{P}$, i.e. $j \in \vec{E}_{I_j} \cap \mathcal{H} \setminus \widehat{P}$ where the degree of $p_H(\cdot) = 2f$.
 - Let $\tilde{p}(\cdot)$ denote the sum of polynomials of corrupt parties P_j s.t. $j \in \vec{E}_{I_j} \cap \mathcal{B}$ where the degree of $\tilde{p}(\cdot) \leq 2f$.
 - $\mathcal{S}$ invokes $\mathcal{F}_{\text{BDKG}}$ with input $(\text{BIAS}, \text{sid}, p'(\cdot))$ where $p'(\cdot) = \tilde{p}(\cdot) + p_H(\cdot)$.

Figure 11: Simulator for our DKG Protocol